



北京化工大学
BEIJING UNIVERSITY OF CHEMICAL TECHNOLOGY



机器学习-回归学习

项德良

xiangdeliang@buct.edu.cn

2022年10月

回归的概念

2

监督学习分为回归和分类

✓ 回归 (Regression、Prediction)

- ✓ 如何预测上海浦东的房价？
- ✓ 未来的股票市场走向？

标签连续

✓ 分类 (Classification)

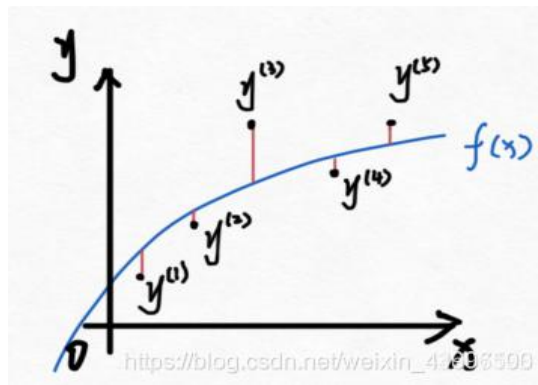
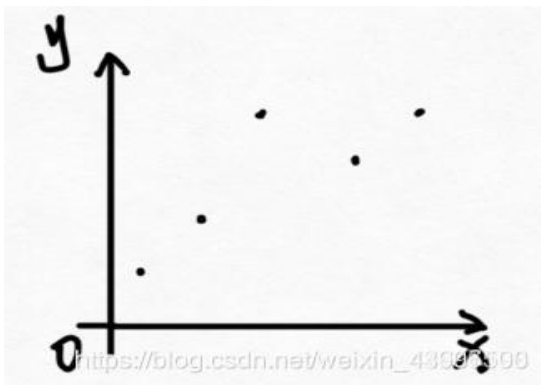
- ✓ 身高1.85m，体重100kg的男人穿什么尺码的T恤？
- ✓ 根据肿瘤的体积、患者的年龄来判断良性或恶性？

标签离散

回归的概念

3

- 回归分析是指一种预测性的建模技术，主要是研究自变量和因变量的关系。通常使用线/曲线来拟合数据点，然后研究如何使曲线到数据点的距离差异最小。



回归问题是机器学习三大基本模型中很重要的一环。

- 回归问题多用来预测一个具体的数值，如预测房价、未来的天气情况等等。例如我们根据一个地区的若干年的PM2.5数值变化来估计某一天该地区的PM2.5值大小，预测值与当天实际数值大小越接近，回归分析算法的可信度越高。

回归的概念

4

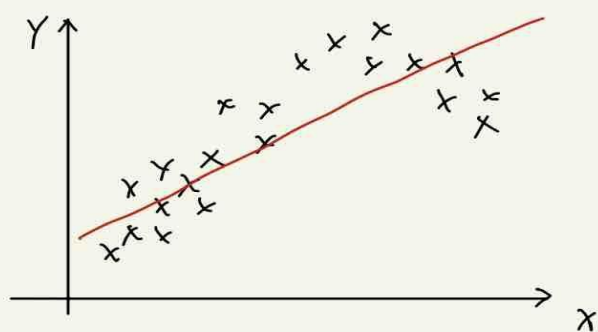
假设给定一组含有 n 个样本的数据集:

$$D = \{(D_i, y_i) | (D_i \in \mathbb{R}^{1 \times d}, y_i \in \mathbb{R}^1, 1 \leq i \leq n)\}$$

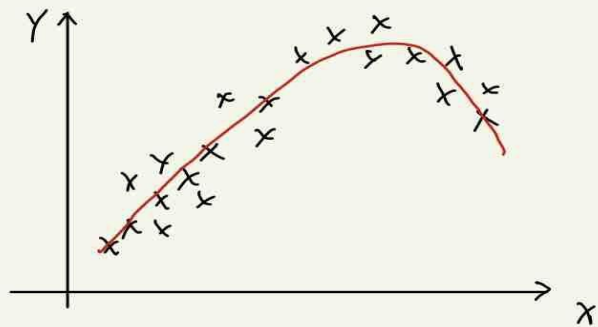
我们去找到一个函数 $f: \mathbb{R}^d \rightarrow \mathbb{R}$, 使得 $f(D_i^T) \approx y_i, (1 \leq i \leq n)$ 。

在上述过程中存在一个底层假设, 即存在一个函数 f^* 可以使得 $y_i = f^*(D_i^T) + \varepsilon_i, \varepsilon_i$ 为噪声。

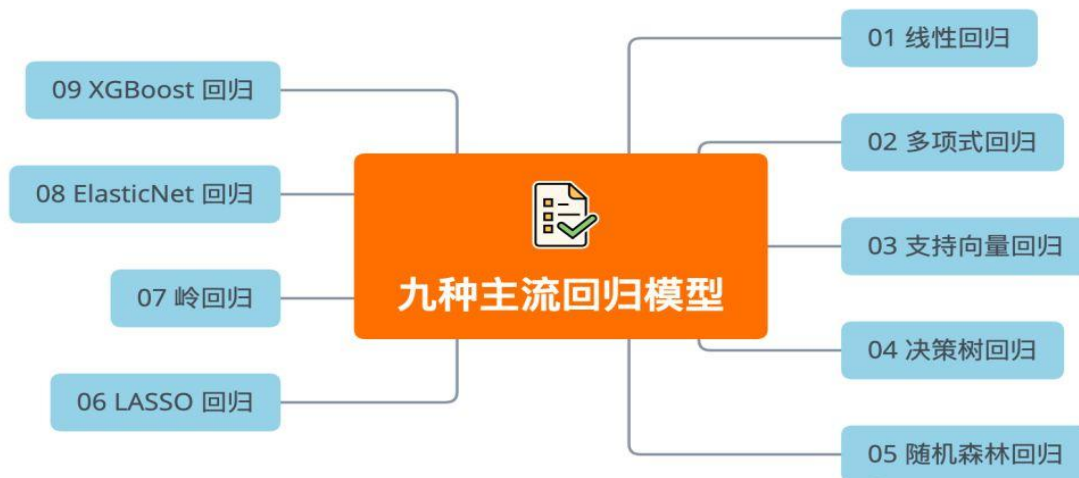
因此可以说: 回归做的事情就是根据数据 D , 寻找到一个函数 $f \in F$ 可以契合函数 f^* 以及其噪声。



linear model



polynomial model



1. 线性模型(Linear Model): $y_w(x) = w_0 + w_1 * x$

2. 多项式模型(Polynomial Model): $y_w(x) = w_0 + w_1 * x + w_2 * x^2 + w_3 * x^3 \dots$

回归的概念

5

面对一个回归问题，我们可简要描述其求解流程：

- ① 选定训练模型，即我们为程序选定一个求解框架，如线性回归模型(Linear Regression)等。
- ② 导入训练集 `train_set`，即给模型提供大量可供学习参考的正确数据。
- ③ 选择合适的学习算法，通过训练集中大量输入输出结果让程序不断优化**输入数据与输出数据间的关联性**，从而提升模型的预测准确度。
- ④ 在训练结束后即可让模型预测结果，我们为程序提供一组新的输入数据，模型根据训练集的学习成果来预测这组输入对应的输出值。

本章目录

6

- 一 线性回归
- 二 多项式回归
- 三 逻辑回归

1. 线性回归

7

01 线性回归

02 梯度下降

03 正则化

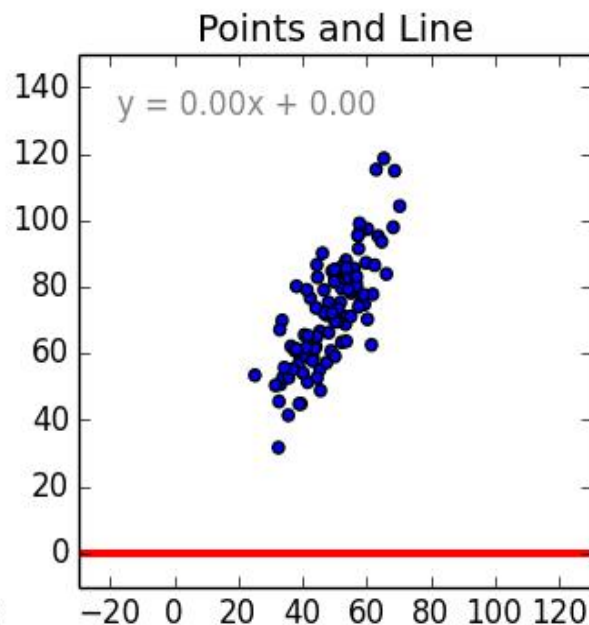
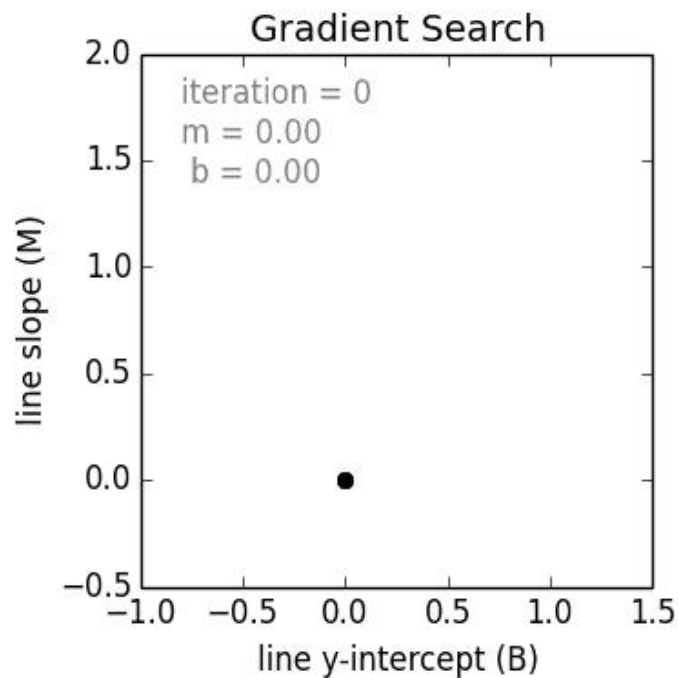
04 回归的评价指标

线性回归-概念

8

线性回归 (Linear Regression)

是一种通过属性的线性组合来进行预测的**线性模型**，其目的是找到一条直线或者一个平面或者更高维的超平面，**使得预测值与真实值之间的误差最小化。**



线性回归-概念

9

给定n个属性 $x=(x_1, x_2, \dots, x_n)$, 其中 x_i 是x在第i个属性上的取值, 线性模型 试图学得一个通过各属性的【线性】组合来进行预测的函数, 即

$$f(x) = \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \beta_0$$

其中 β_i 和 β_0 为参数, 参数学得之后, 模型就可以确定了。

注意: 这里x是已知的, 未知数是 β_i 和 β_0 , 所以问题来了, 线性, 指的是谁“线性”?

(这个问题值得注意, 因为一般写线性回归的内容中对这个都忽略不写)

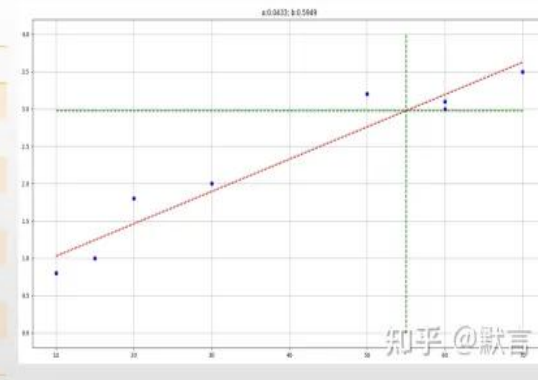
其实线性的对象是参数 β_i 和 β_0 , 而不是x, x可以是非线性的。对x的函数(可以是非线性)进行线性组合, 可以构建出更多有用的函数, 这些x的函数称为“基函数”, 其中多项式是最常见的基函数, 基函数为 $\phi_j(x) = x^j$, 它可用于拟合曲线(非线性的数据)。

线性模型形式简单, 直观、易于理解, 可解释性强, 例如 w_i 的值就直观的代表了各属性在预测中的重要程度。

一元线性回归

$$y = ax + b$$

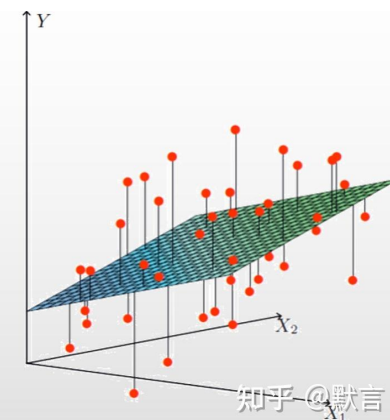
房屋面积(m^2)	租赁价格(1000 ¥)
10	0.8
15	1
20	1.8
30	2
50	3.2
60	3
60	3.1
70	3.5



多元线性回归

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

房屋面积	房间数量	租赁价格
10	1	0.8
15	1	1.0
20	1	1.8
30	1	2.0
50	2	3.2
60	1	3.0
60	2	3.1
70	2	3.5
.....		



线性回归-符号约定

10

m 代表训练集中样本的数量

n 代表特征的数量

x 代表特征/输入变量

y 代表目标变量/输出变量

(x, y) 代表训练集中的样本

$(x^{(i)}, y^{(i)})$ 代表第 i 个观察样本

h 代表学习算法的解决方案或函数也称为假设 (**hypothesis**)

$\hat{y} = h(x)$, 代表预测的值

建筑面积	总层数	楼层	实用面积	房价
143.7	31	10	105	36200
162.2	31	8	118	37000
199.5	10	10	170	42500
96.5	31	13	74	31200
.....				

$x^{(i)}$ 是特征矩阵中的第 i 行, 是一个**向量**。

上图的: $x^{(2)} = \begin{bmatrix} 162.2 \\ 31 \\ 8 \\ 118 \end{bmatrix}$ $y^{(2)} = 37000$

$x_j^{(i)}$ 代表特征矩阵中第 i 行的第 j 个特征

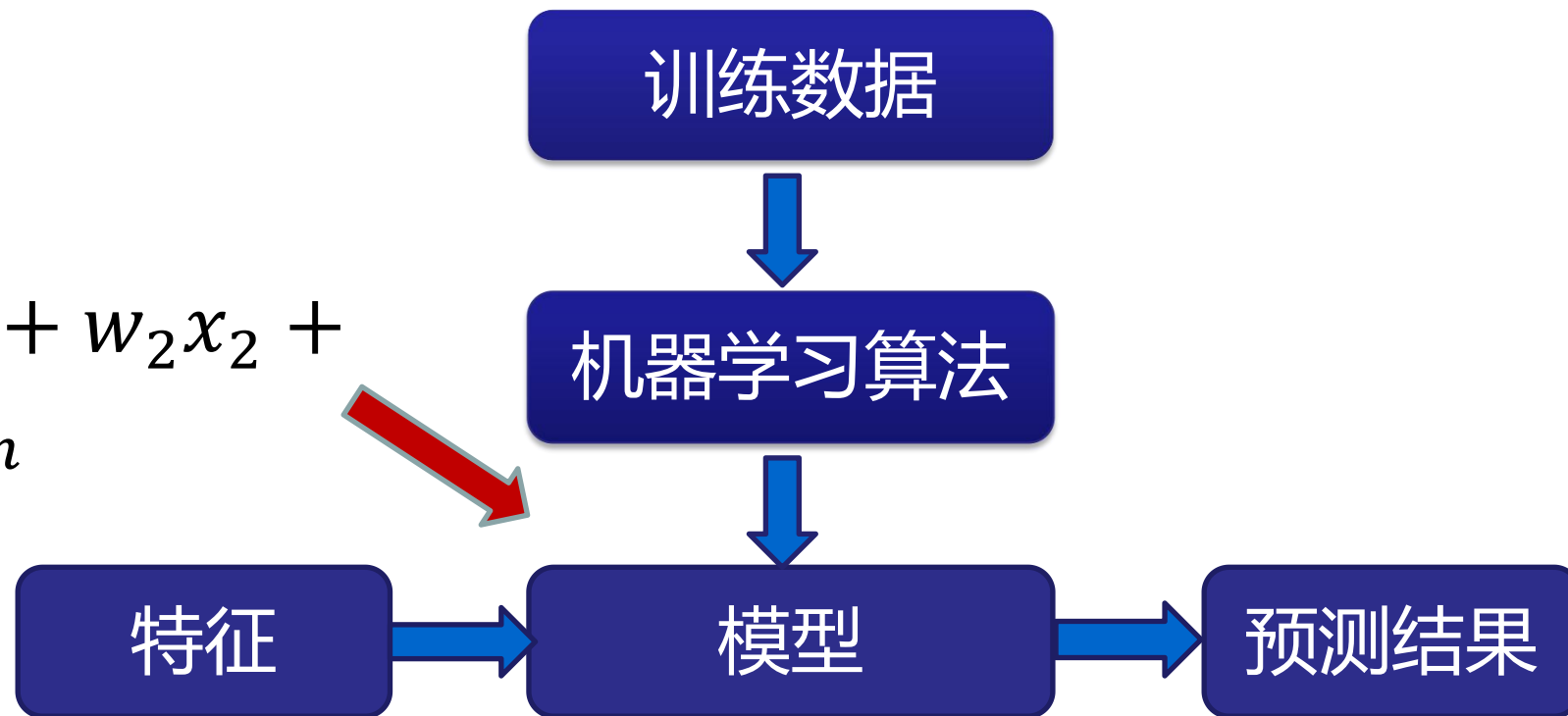
上图的 $x_2^{(2)} = 31, x_3^{(2)} = 8$

线性回归-算法流程

11

x 和 y 的关系

$$h(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$



可以设 $x_0 = 1$

则: $h(x) = w_0x_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = w^T X$

注意: 若表达式 $h(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + b$, 则 b 可以融入到 w_0

线性回归-算法流程

12

$$h(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

损失函数采用平方和损失：

$$l(x^{(i)}) = \frac{1}{2} (h(x^{(i)}) - y^{(i)})^2$$

?

要找到一组 $w(w_0, w_1, w_2, \dots, w_n)$

, 使得 $J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$
(残差平方和) 最小

损失函数 (Loss Function) 度量**单样本**预测的错误程度，损失函数值越小，模型就越好。常用的损失函数包括：0-1损失函数、平方损失函数、绝对损失函数、对数损失函数等。

代价函数 (Cost Function) 度量**全部样本集**的平均误差。常用的代价函数包括均方误差、均方根误差、平均绝对误差等。

目标函数 (Object Function) 代价函数和正则化函数，最终要优化的函数。

线性回归-算法流程

13

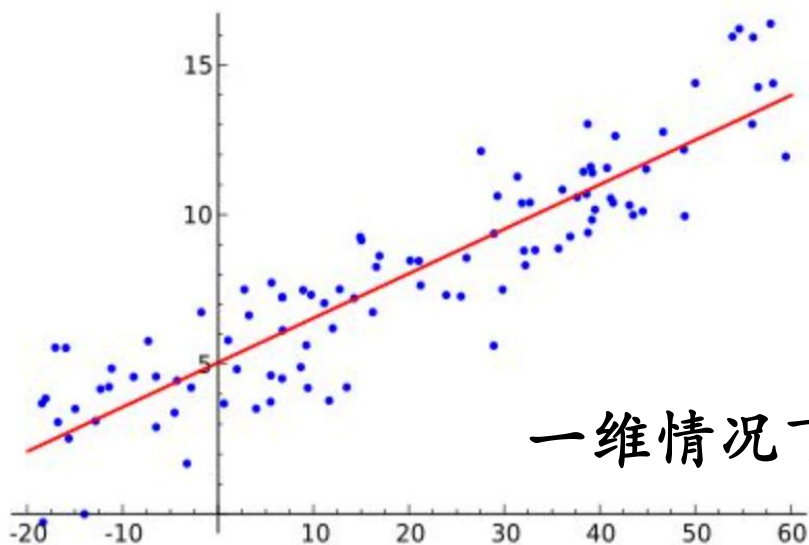
独立同分布条件 (I.I.D) 的训练数据集

$$\mathbf{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$$

对每个样本，模型与标注之间存在误差 ε_i

$$y_i = \hat{y}(\mathbf{x}_i, \mathbf{w}) + \varepsilon_i = \mathbf{w}^T \bar{\mathbf{x}}_i + \varepsilon_i$$

$$\varepsilon_i \sim N(0, \sigma_\varepsilon^2)$$



一维情况下，线性回归表示的例子

中心极限定理

线性回归-算法流程

14

通过最大似然
原理导出线性
回归的学习算
法

y_i 的概率密度函数

$$p_y(y_i | \mathbf{w}) = \frac{1}{(2\pi\sigma_\varepsilon^2)^{\frac{1}{2}}} \exp \left[-\frac{1}{2\sigma_\varepsilon^2} (y_i - \hat{y}(\mathbf{x}_i, \mathbf{w}))^2 \right]$$

所有样本的标注值表示为向量

$$\mathbf{y} = [y_1, y_2, y_3, \dots, y_N]^T$$

由样本集的I.I.D性得似然函数

$$\begin{aligned} p_y(\mathbf{y} | \mathbf{w}) &= \prod_{i=1}^N p_y(y_i | \mathbf{w}) \\ &= \frac{1}{(2\pi\sigma_\varepsilon^2)^{\frac{N}{2}}} \exp \left[-\frac{1}{2\sigma_\varepsilon^2} \sum_{i=1}^N (y_i - \hat{y}(\mathbf{x}_i, \mathbf{w}))^2 \right] \end{aligned}$$

线性回归-算法流程

15

取对数似然为

$$\log p_y(\mathbf{y}|\mathbf{w}) = -\frac{N}{2}\log(2\pi\sigma_\varepsilon^2) - \frac{1}{2\sigma_\varepsilon^2} \sum_{i=1}^N (y_i - \hat{y}(\mathbf{x}_i, \mathbf{w}))^2$$

ML导出线性回归的学习算法

对数似然函数最大，对应平方误差和最小

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N (y_i - \hat{y}(\mathbf{x}_i, \mathbf{w}))^2$$

重写误差平方和

$$\begin{aligned} J(\mathbf{w}) &= \frac{1}{2} \sum_{n=1}^N \varepsilon_i^2 = \frac{1}{2} \sum_{i=1}^N (y_i - \mathbf{w}^T \bar{\mathbf{x}}_i)^2 \\ &= \frac{1}{2} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) \\ &= \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 \end{aligned}$$

其中

$$\mathbf{X} = \begin{bmatrix} \bar{\mathbf{x}}_1^T \\ \bar{\mathbf{x}}_2^T \\ \vdots \\ \bar{\mathbf{x}}_N^T \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1K} \\ 1 & x_{21} & \cdots & x_{2K} \\ \vdots & \vdots & \cdots & \vdots \\ 1 & x_{N1} & \cdots & x_{NK} \end{bmatrix}$$

线性回归-最小二乘法(LSM)

16

要找到一组 $w(w_0, w_1, w_2, \dots, w_n)$ ，使得 $J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2$

(残差平方和) 最小，即最小化： $\frac{\partial J(w)}{\partial w}$

将向量表达形式转为矩阵表达形式，则有 $J(w) = \frac{1}{2} (Xw - Y)^2$ ，其中 X 为 m 行 $n + 1$ 列的矩阵（ m 为样本个数， n 为特征个数）， w 为 $n + 1$ 行 1 列的矩阵（包含了 w_0 ）， Y 为 m 行 1 列的矩阵，则 $J(w) = \frac{1}{2} (Xw - Y)^2 = \frac{1}{2} (Xw - Y)^T (Xw - Y)$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & x_3^{(2)} & \dots & x_n^{(2)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & x_3^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(m)} \end{bmatrix}$$

需要用到向量平方的性质：

$$\sum_i z_i^2 = z^T z$$

线性回归-最小二乘法(LSM)

17

为最小化, 接下来对 $J(w)$ 偏导,

$$\frac{\partial J(w)}{\partial w} = \frac{1}{2} \frac{\partial}{\partial w} (Xw - Y)^T (Xw - Y) = \frac{1}{2} \frac{\partial}{\partial w} (w^T X^T X w - Y^T X w - w^T X^T Y + Y^T Y)$$

由于中间两项互为转置:

$$\frac{\partial J(w)}{\partial w} = \frac{1}{2} \frac{\partial}{\partial w} (w^T X^T X w - 2w^T X^T Y + Y^T Y) = \frac{1}{2} (2X^T X w - 2X^T Y + 0)$$

$$= X^T X w - X^T Y$$

$$\text{令 } \frac{\partial J(w)}{\partial w} = 0,$$

$$\text{则有 } w = (X^T X)^{-1} X^T Y$$

需要用到以下几个矩阵的求导法则:

$$\frac{dX^T X}{dX} = 2X \quad \frac{dAX}{dX} = A^T$$

$$\frac{\partial X^T A X}{\partial X} = (A + A^T)X, \text{ 若 } A \text{ 为对称阵, } \frac{\partial X^T A X}{\partial X} = 2AX$$

线性回归-最小二乘法(LSM)

18

1. LSM仅适用于线性回归模型，不可乱用；
2. 在使用正规方程求解时无需进行特征缩放；
3. 若 $X^T X$ 为奇异矩阵则无法求其逆矩阵，使用正则化的方法可以保证矩阵可逆；
4. 使用LSM时应该注意当特征数量规模大于10000时， $(X^T X)^{-1}$ 求其逆矩阵的时间复杂度会很高，不宜用正规方程求解。

1. 线性回归

19

01 线性回归

02 梯度下降

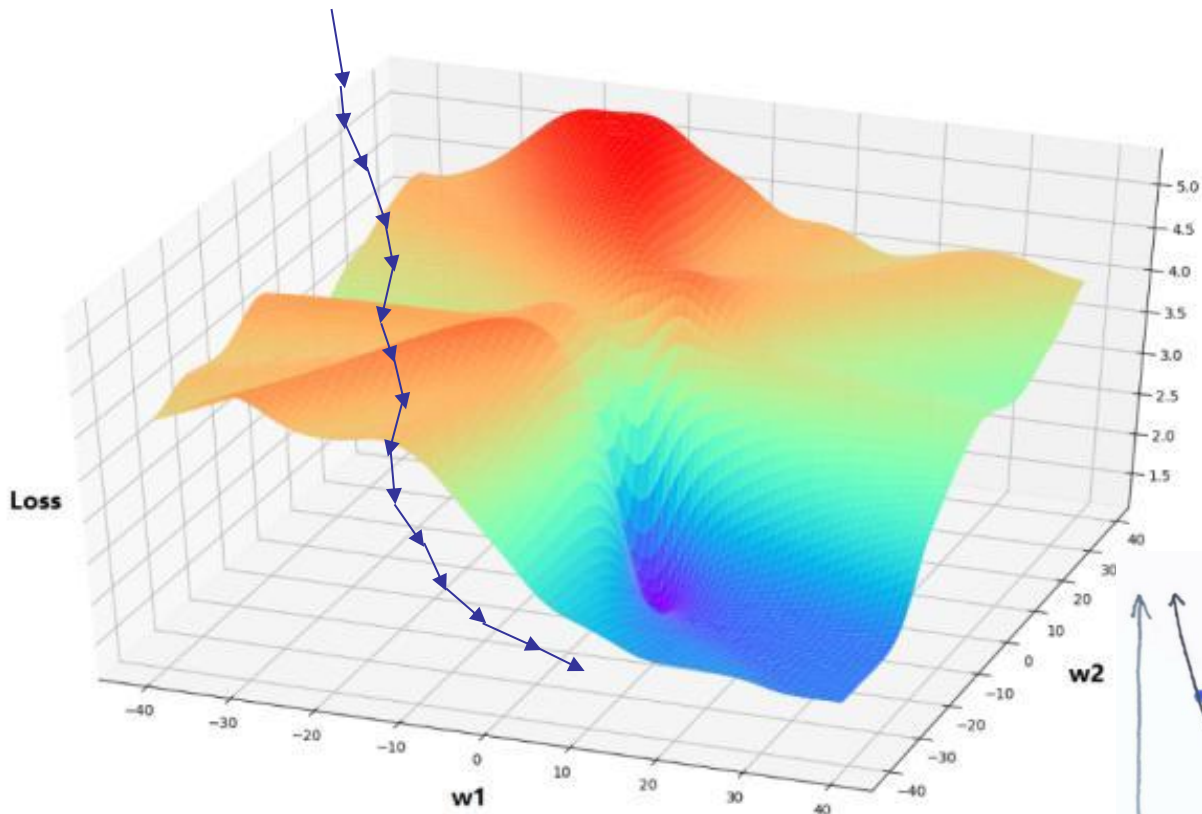
03 正则化

04 回归的评价指标

梯度下降

20

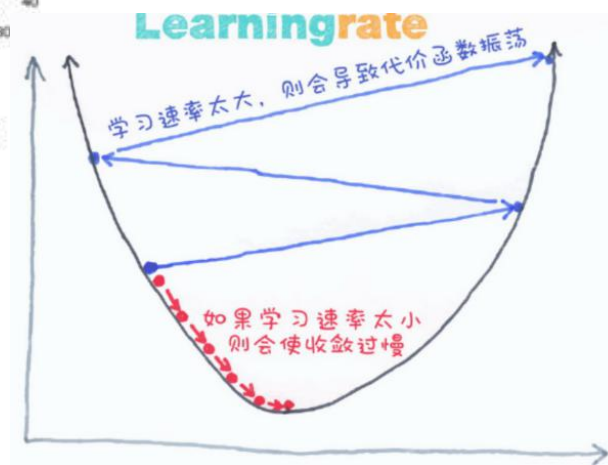
首先，我们有一个可微分的函数。这个函数就代表着一座山。我们的目标就是找到这个函数的最小值，也就是山底。根据之前的场景假设，最快的下山的方式就是找到当前位置最陡峭的方向，然后沿着此方向向下走，对应到函数中，就是找到给定点的梯度，然后朝着梯度相反的方向，就能让函数值下降的最快！因为梯度的方向就是函数之变化最快的方向



学习率

α

步长



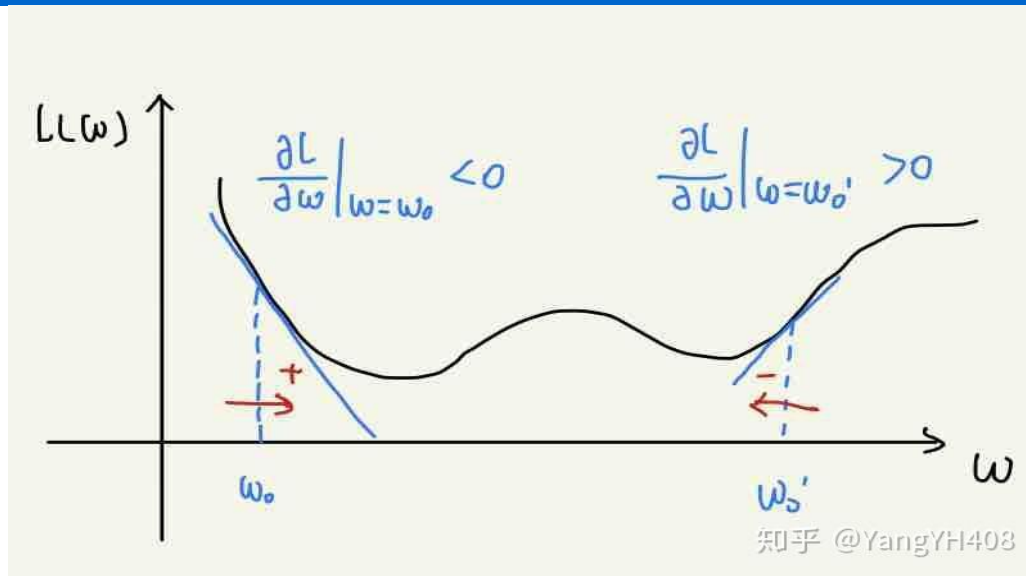
梯度下降

21

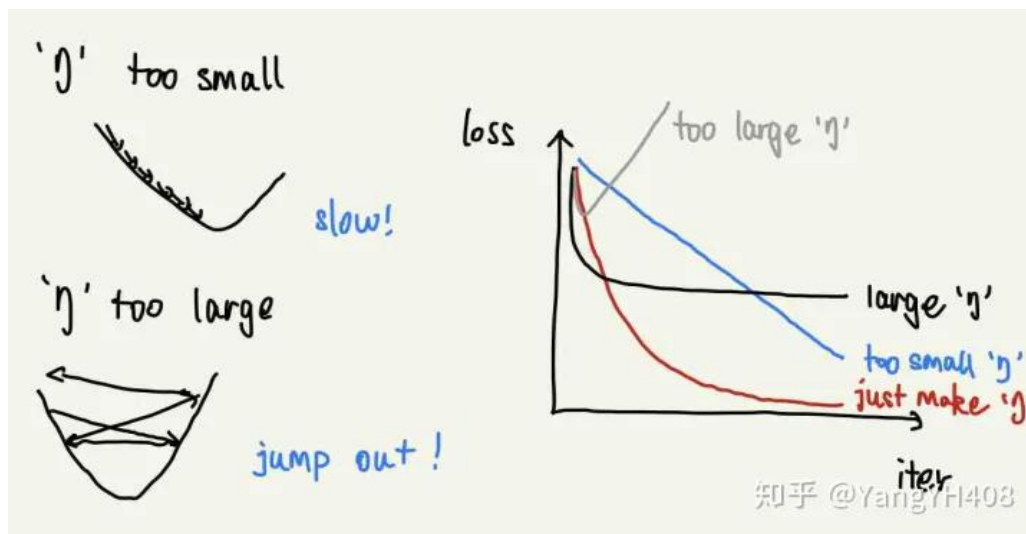
由导数知识我们不难发现，要使损失函数的值减小，我们只需让回归系数向与当前位置偏导数符号相反的方向更新即可

$$w_1 \leftarrow w_0 - \eta \frac{\partial L}{\partial w} \Big|_{w=w_0}$$

其中，超参数 η 代表学习速率(learning_rate)，即单次更新步长。 η 值的选择需谨慎，如果太小更新速率太慢则很难到达；如果太大则容易直接越过极值点。



知乎 @YangYH403



知乎 @YangYH403

梯度下降的三种形式

22

批量梯度下降 (Batch Gradient Descent, BGD)

梯度下降的每一步中，都用到了所有的训练样本

随机梯度下降 (Stochastic Gradient Descent, SGD)

梯度下降的每一步中，用到一个样本，在每一次计算之后便更新参数，而不需要首先将所有的训练集求和

小批量梯度下降 (Mini-Batch Gradient Descent, MBGD)

梯度下降的每一步中，用到了一定批量的训练样本

梯度下降的三种形式

23

批量梯度下降 (Batch Gradient Descent)

梯度下降的每一步中，都用到了所有的训练样本

参数更新

学习率

梯度

$$w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m \left((h(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)} \right)$$

(同步更新 w_j , $(j=0,1,...,n)$)

通过下面这个公式可以看出，如果要求出梯度，每一项都要对所有的样本进行一次计算。这种计算方式一定能精确地求出最优梯度。但如果样本量 m 比较大的时候，计算梯度那是相当耗时的。因此，基于这个问题就有了随机梯度下降。

梯度下降的三种形式

24

随机梯度下降 (Stochastic Gradient Descent)

推导 $w = w - \alpha \cdot \frac{\partial J(w)}{\partial w}$ $h(x) = w^T X = w_0 x_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n$

$$J(w) = \frac{1}{2} (h(x^{(i)}) - y^{(i)})^2$$

$$\frac{\partial}{\partial w_j} J(w) = \frac{\partial}{\partial w_j} \frac{1}{2} (h(x^{(i)}) - y^{(i)})^2 = 2 \cdot \frac{1}{2} (h(x^{(i)}) - y^{(i)}) \cdot \frac{\partial}{\partial w_j} (h(x^{(i)}) - y^{(i)})$$

$$= (h(x^{(i)}) - y^{(i)}) \cdot \frac{\partial}{\partial w_j} \left(\sum_{i=0}^n (w_i x_i^{(i)} - y^{(i)}) \right)$$

$$= (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

梯度下降的三种形式

25

随机梯度下降 (Stochastic Gradient Descent)

梯度下降的每一步中，用到一个样本，在每一次计算之后便更新参数，而不需要首先将所有的训练集求和

参数更新

$$w_j := w_j - \alpha (h(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad \alpha = \frac{t_0}{i_iters + t_1}$$

(同步更新 w_j , $(j=0,1,...,n)$)

每次只对一个样本进行求梯度，这样也就不用除以m了。基于这样的想法就出现了随机选择样本求梯度的方法，既然不确定这个搜索方向是不是梯度方向，那么学习率 α 这个参数就更加重要了，如果一直取个不变的学习率，很有可能到达最优解之后还会跳出去。因此，在实际的使用过程中，在随机梯度下降法中需要让学习率逐渐递减。

梯度下降的三种形式

26

小批量梯度下降 (Mini-Batch Gradient Descent)

梯度下降的每一步中，用到了一定批量的训练样本

每计算常数 b 次训练实例，便更新一次参数 w

参数更新

$$w_j := w_j - \alpha \frac{1}{b} \sum_{k=i}^{i+b-1} (h(x^{(k)}) - y^{(k)}) x_j^{(k)}$$

(同步更新 w_j , ($j=0,1,\dots,n$))

$b=1$ (随机梯度下降,SGD)
 $b=m$ (批量梯度下降,BGD)
 $b=batch_size$, 通常是2的指数倍, 常见有32,64,128等。
(小批量梯度下降,MBGD)

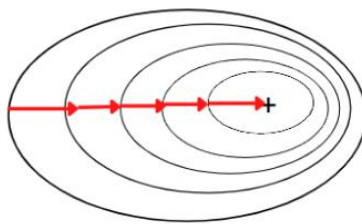
梯度下降的三种形式

它产生稳定的误差梯度和稳定的收敛。一些缺点是稳定的误差梯度有时会导致收敛状态不是模型所能达到的最佳状态。

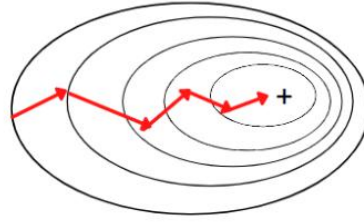
27

缩写	名称	原理	优点	缺点
BGD	批量梯度下降	用所有样本去更新梯度	能达到最优解	耗时
SGD	随机梯度下降法	随机选择一个样本更新参数	速度快	局部最优解（凸函数时达到全局最优解），对特征敏感
MGBD	小批量梯度下降	选择一小批数据更新参数	相对于BGD更快，比SGD更准	

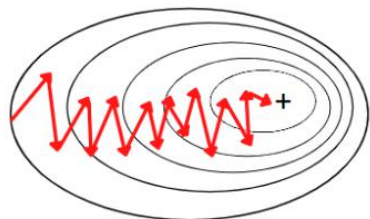
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



更新的频率也会产生噪声梯度，这可能导致错误率波动而不是逐渐下降。

梯度下降法找到的一定是下降最快的方向吗？

不一定，它只是目标函数在当前的点的切平面上下降最快的方向。

在实际执行期中，**牛顿方向（考虑海森矩阵）**才一般被认为是下降最快的方向，可以达到超线性的收敛速度。梯度下降类的算法的收敛速度一般是线性甚至次线性的（在某些带复杂约束的问题）

梯度下降与最小二乘法比较

28

梯度下降：需要选择学习率 α ，需要多次迭代，当特征数量 n 大时也能较好适用，适用于各种类型的模型。

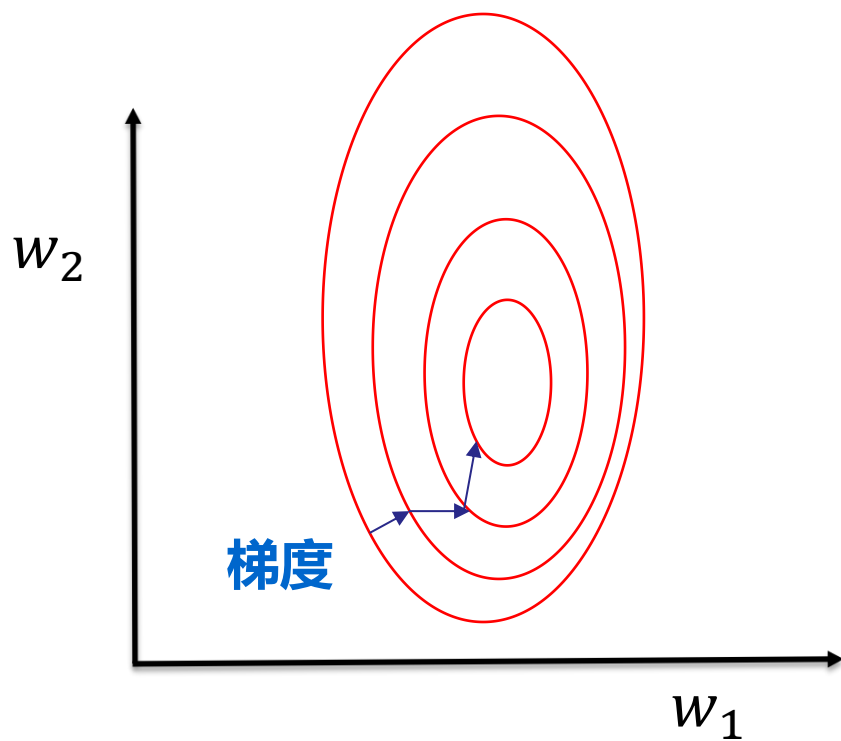
最小二乘法：不需要选择学习率 α ，一次计算得出，需要计算 $(X^T X)^{-1}$ ，如果特征数量 n 较大则运算代价大，因为矩阵逆的计算时间复杂度为 $O(n^3)$ ，通常来说当 n 小于10000 时还是可以接受的，只适用于线性模型，不适合逻辑回归模型等其他模型。

数据归一化/标准化

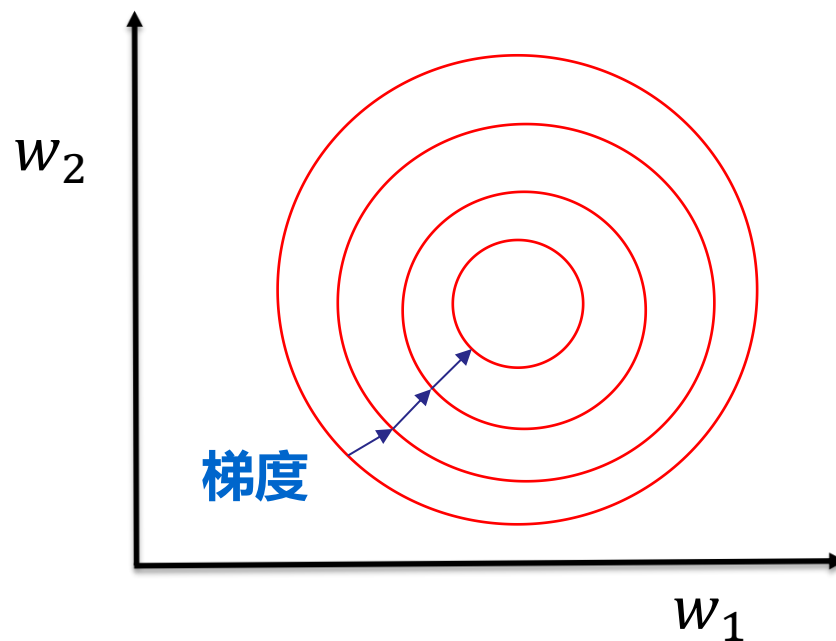
29

为什么要标准化/归一化？

提升模型精度：不同维度之间的特征在数值上有一定比较性，可以大大提高分类器的准确性。



加速模型收敛：最优解的寻优过程明显会变得平缓，更容易正确的收敛到最优解。



数据归一化/标准化

30

归一化（最大 - 最小规范化）

$$x^* = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

将数据映射到[0,1]区间

数据归一化的目的是使得各特征对目标变量的影响一致，会将特征数据进行伸缩变化，所以数据归一化是会改变特征数据分布的。

Z-Score标准化

$$x^* = \frac{x - \mu}{\sigma}$$

处理后的数据均值为0，方差为1

数据标准化为了不同特征之间具备可比性，经过标准化变换之后的特征数据分布没有发生改变。

就是当数据特征取值范围或单位差异较大时，最好是做一下标准化处理。

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2$$
$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

数据归一化/标准化

31

需要做数据归一化/标准化

线性模型，如基于距离度量的模型包括KNN(K近邻)、K-means聚类、感知机和SVM。另外，线性回归类的几个模型一般情况下也是需要做数据归一化/标准化处理的。

不需要做数据归一化/标准化

决策树、基于决策树的Boosting和Bagging等集成学习模型对于特征取值大小并不敏感，如随机森林、XGBoost、LightGBM等树模型，以及朴素贝叶斯，以上这些模型一般不需要做数据归一化/标准化处理。

3. 正则化

32

01 线性回归

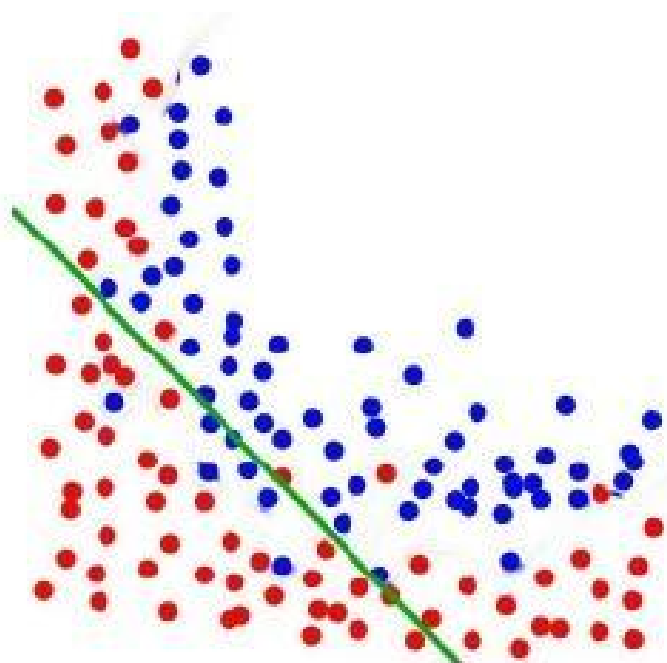
02 梯度下降

03 正则化

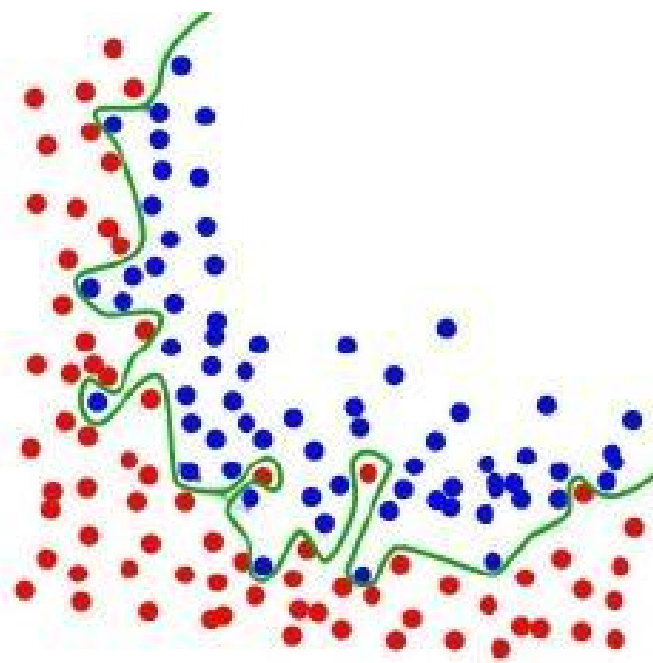
04 回归的评价指标

过拟合和欠拟合

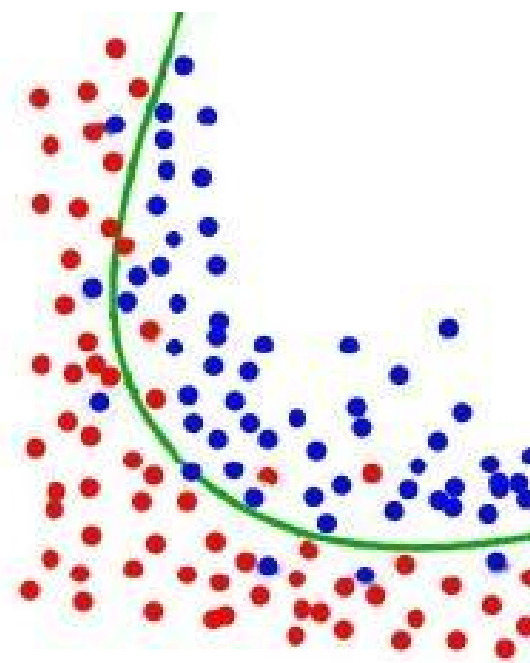
33



欠拟合



过拟合



正合适

过拟合的处理

34

1. 获得更多的训练数据

使用更多的训练数据是解决过拟合问题最有效的手段，因为更多的样本能够让模型学习到更多更有效的特征，减小噪声的影响。

2. 降维

即丢弃一些不能帮助我们正确预测的特征。可以是手工选择保留哪些特征，或者使用一些模型选择的算法来帮忙（例如PCA）。

3. 正则化

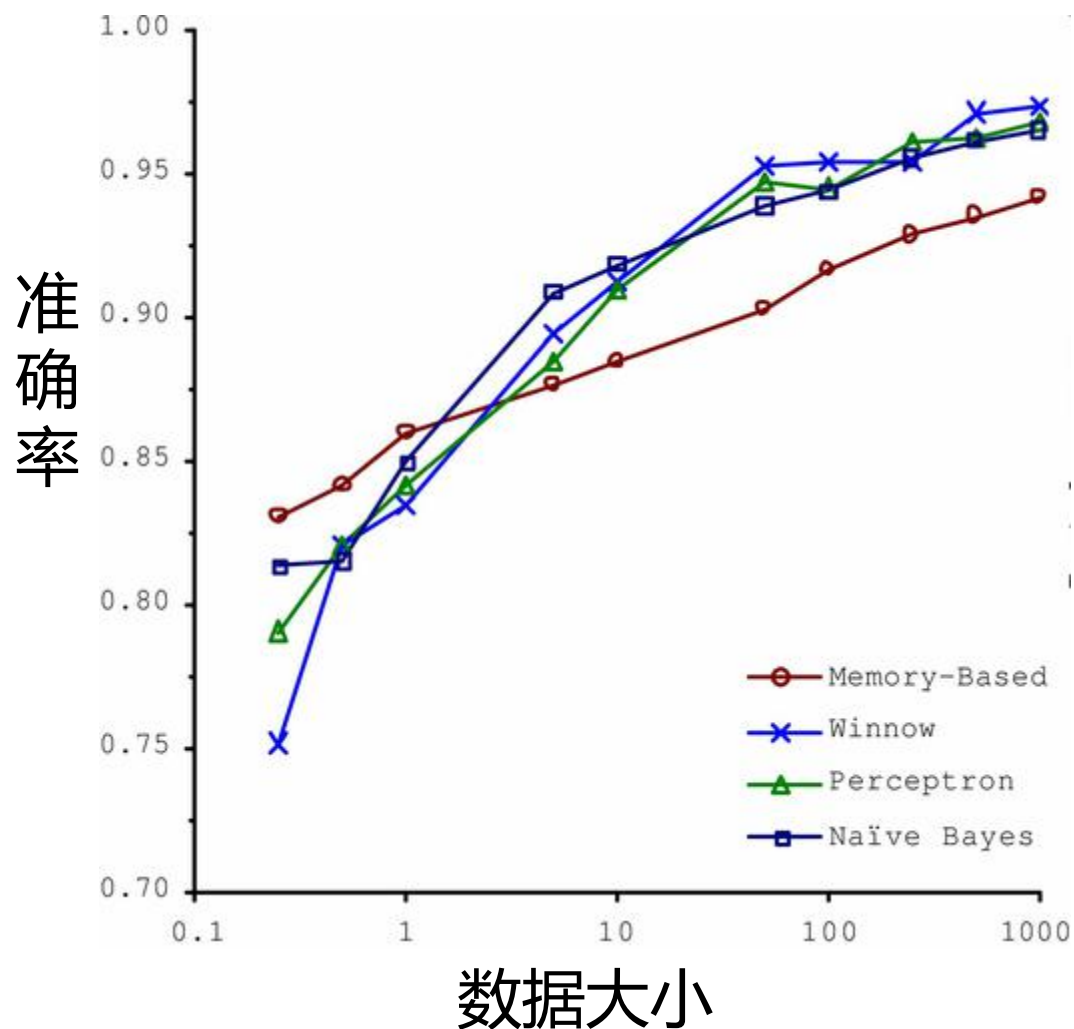
正则化(regularization)的技术，保留所有的特征，但是减少参数的大小（magnitude），它可以改善或者减少过拟合问题。

4. 集成学习方法

集成学习是把多个模型集成在一起，来降低单一模型的过拟合风险。

数据决定一切

35



通过这张图可以看出，各种不同算法在输入的数据量达到一定级数后，都有相近的高准确度。于是诞生了机器学习界的名言：

成功的机器学习应用不是拥有最好的算法，而是拥有最多的数据！

欠拟合的处理

36

1. 添加新特征

当特征不足或者现有特征与样本标签的相关性不强时，模型容易出现欠拟合。通过挖掘组合特征等新的特征，往往能够取得更好的效果。

2. 增加模型复杂度

简单模型的学习能力较差，通过增加模型的复杂度可以使模型拥有更强的拟合能力。例如，在线性模型中添加高次项，在神经网络模型中增加网络层数或神经元个数等。

3. 减小正则化系数

正则化是用来防止过拟合的，但当模型出现欠拟合现象时，则需要有针对性地减小正则化系数。

正则化

37

L1正则化: $J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n |w_j|$, Lasso Regression (Lasso回归)

L2正则化: $J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n w_j^2$, Ridge Regression (岭回归)

Elastic Net: $J(w) = \frac{1}{2} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2 + \lambda(\rho \cdot \sum_{j=1}^n |w_j| + (1 - \rho) \cdot \sum_{j=1}^n w_j^2)$
(弹性网络)



其中:

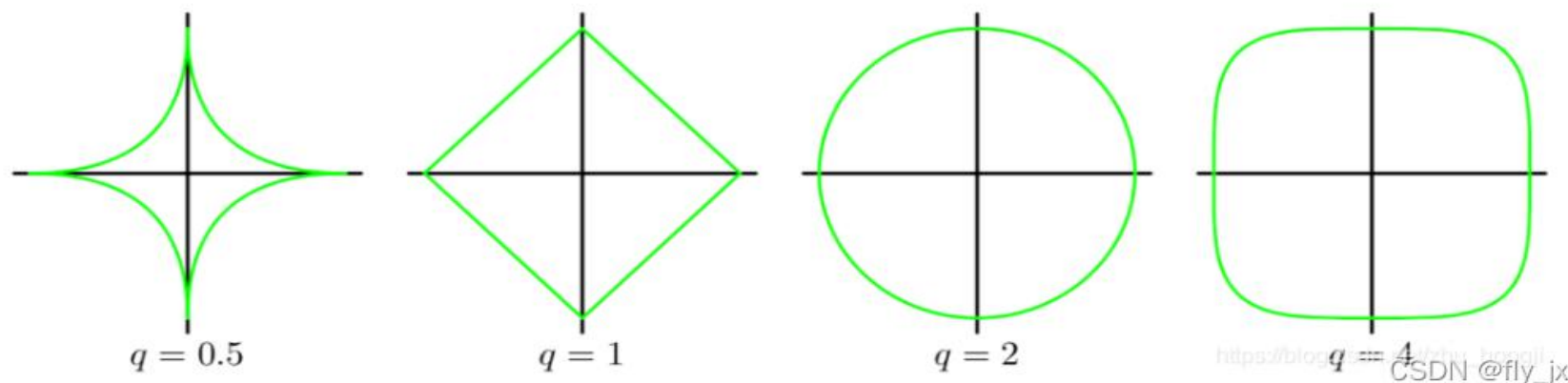
- λ 为正则化系数, 调整正则化项与训练误差的比例, $\lambda > 0$ 。
- $1 \geq \rho \geq 0$ 为比例系数, 调整L1正则化与L2正则化的比例。

正则化

38

正则化一般分为L1和L2正则，也叫L1范数，L2范数。

- 1-范数： $\|x\|_1 = \sum_{i=1}^N |x_i|$ ，即向量元素绝对值之和。
- 2-范数： $\|x\|_2 = \sqrt{\sum_{i=1}^N x_i^2}$ ，又叫欧几里得(Euclid)范数，是很常见的向量长度计算公式。即向量元素绝对值平方和再进行开方。
- q-范数： $\|x\|_q = (\sum_{i=1}^N |x_i|^q)^{\frac{1}{q}}$ ，向量元素绝对值的q次幂的累加和再 $\frac{1}{q}$ 次幂。

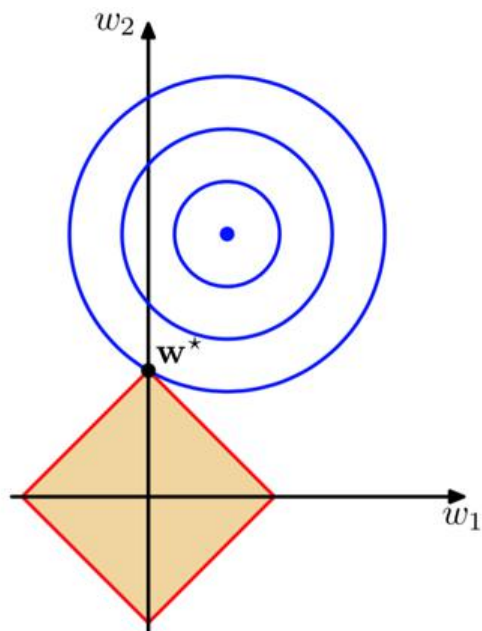


L1正则化的原则是样本特性是由少数重要特征决定的，因此将不重要的特征系数归零，只考虑重要特征的系数，所以L1正则化会产生稀疏矩阵。这也是L1正则解决过拟合的原因。

L2正则化的原则，降低系数的值，提高模型的泛化性。如果模型在训练集上拟合的很好，且模型参数的值都很小，这样模型就能在各种数据集上具备很好的泛化性。

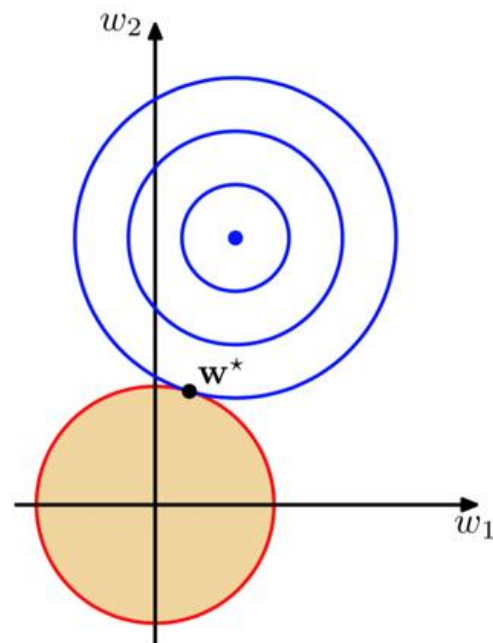
正则化

39



L1正则化是指在损失函数中加入权值向量 w 的绝对值之和, L1的功能是使权重稀疏

L1正则化可以产生稀疏模型



在损失函数中加入权值向量 w 的平方和, L2的功能是使权重平滑。

L2正则化可以防止过拟合

图上面中的蓝色轮廓线是没有正则化损失函数的等高线, 中心的蓝色点为最优解, 左图、右图分别为L1、L2正则化给出的限制。

可以看到在正则化的限制之下, L2正则化给出的最优解 w^* 是使解更加靠近原点, 也就是说L2正则化能降低参数范数的总和。

L1正则化给出的最优解 w^* 是使解更加靠近某些轴, 而其它的轴则为0, 所以L1正则化能使得到的参数稀疏化。

4. 回归的评价指标

40

01 线性回归

02 梯度下降

03 正则化

04 回归的评价指标

回归的评价指标

41

均方误差 (Mean Square Error,MSE)

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

均方根误差 RMSE(Root Mean Square Error, RMSE)

$$\text{RMSE}(y, \hat{y}) = \sqrt{\frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2}$$

平均绝对误差 (Mean Absolute Error, MAE)

$$\text{MAE}(y, \hat{y}) = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - \hat{y}^{(i)}|$$

其中, $y^{(i)}$ 和 $\hat{y}^{(i)}$ 分别表示第 i 个样本的真实值和预测值, m 为样本个数。

回归的评价指标

42

R方 [*RSquared(r2score)*]

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^m (y^{(i)} - \bar{y})^2} = \frac{SSR}{SST}$$

$$= 1 - \frac{SSE}{SST}$$

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2 / m}{\sum_{i=1}^m (y^{(i)} - \bar{y})^2 / m}$$

$$= 1 - \frac{MSE}{Var}$$

越接近于1,说明模型拟合得越好

其中, $y^{(i)}$ 和 $\hat{y}^{(i)}$ 分别表示第*i*个样本的真实值和预测值, m 为样本个数。

$$SSR = \sum_{i=1}^m (\hat{y}^{(i)} - \bar{y})^2$$

$$SSE = \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

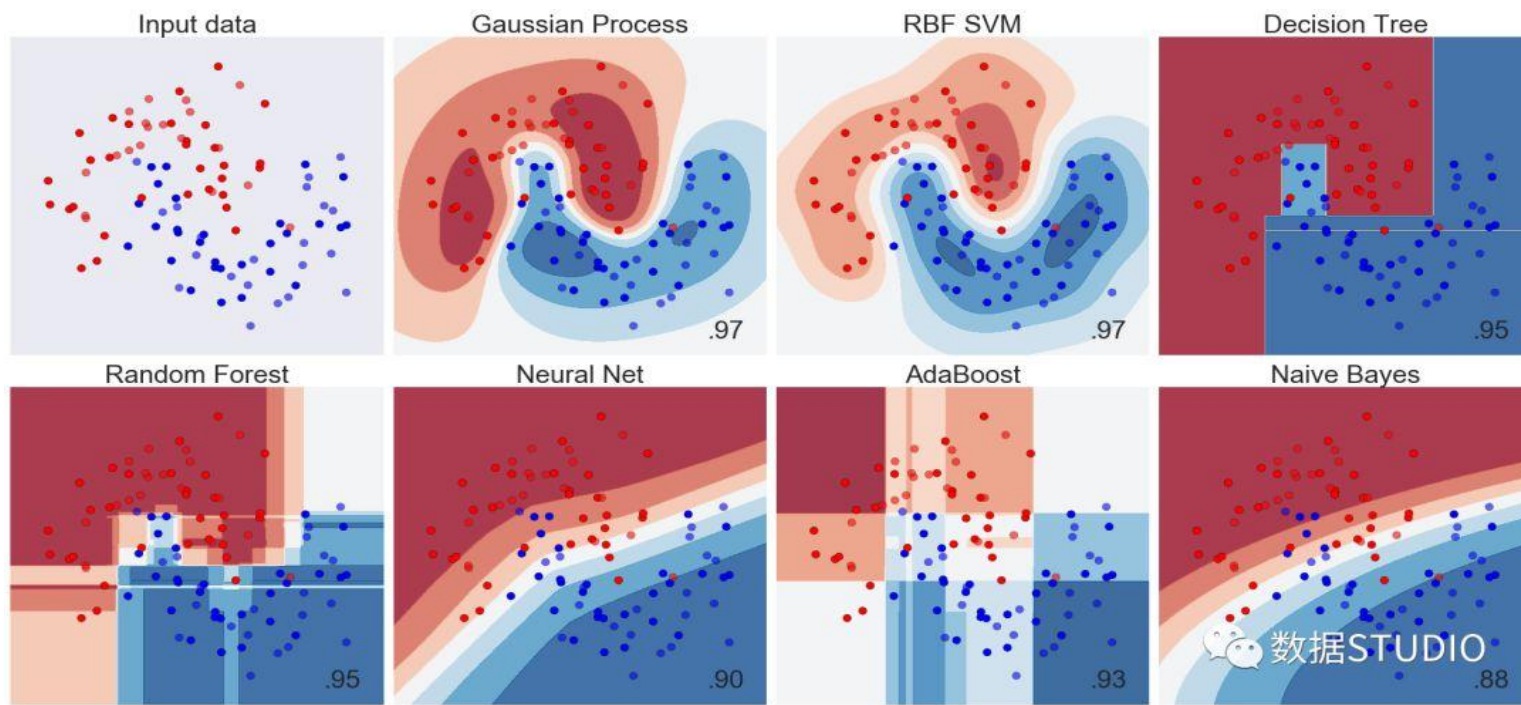
$$SST = \sum_{i=1}^m (y^{(i)} - \bar{y})^2$$

- [1] Andrew Ng. Machine Learning[EB/OL]. StanfordUniversity,2014.<https://www.coursera.org/course/ml>
- [2] 李航. 统计学习方法[M]. 北京: 清华大学出版社,2019.
- [3] 周志华. 机器学习[M]. 北京: 清华大学出版社,2016.
- [4] WEINBERGER K. Distance metric learning for large margin nearest neighbor classification[J]. Advances in Neural Information Processing Systems, 2006, 18.
- [5] HOERL A E, KENNARD R W. Ridge regression: applications to nonorthogonal problems[J]. Technometrics, 1970, 12(1): 69–82.
- [6] TIBSHIRANI R. Regression selection and shrinkage via the lasso[J]. Journal of the Royal Statistical Society Series B, 1996, 58(1): 267–288.
- [7] TIBSHIRANI R, BICKEL P, RITOV Y, et al. Least absolute shrinkage and selection operator[J]. Software: <http://www.stat.stanford.edu/tibs/lasso.html>, 1996.

2. 多项式回归

44

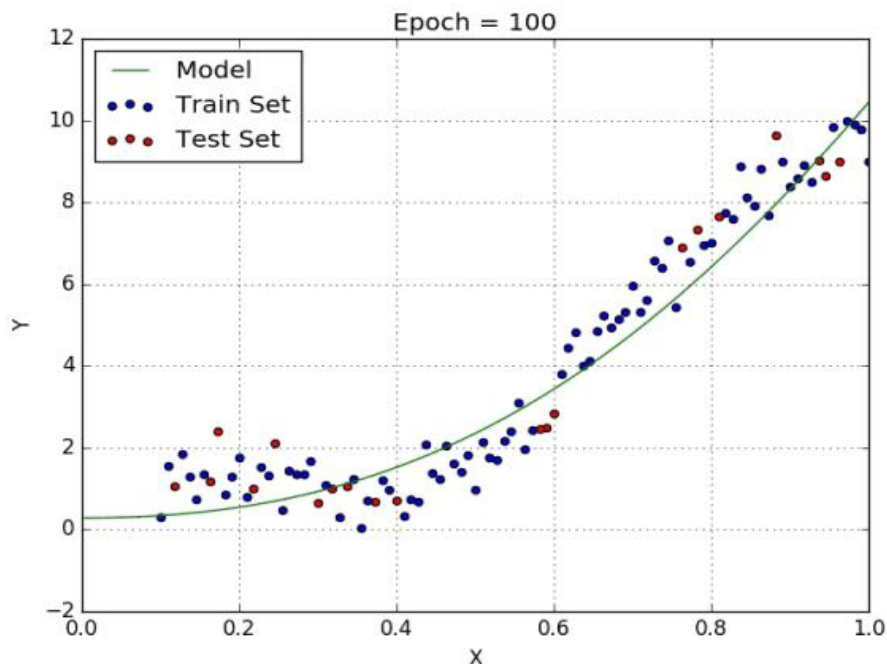
- 在使用线性回归时，除了遇到以上问题（数据中存在多重共线性、数据维度过高），还会遇到数据并不总是线性的，若此时仍坚持用线性模型去拟合非线性数据，其结果往往是很糟糕。庆幸的是除了更换更加复杂的非线性模型外，仍可以使用线性回归方法来拟合非线性数据，只不过在此之前，需要对输入数据进行处理。
- 多项式回归是一种通过增加自变量上的次数，而将数据映射到高维空间的方法，从而提高模型拟合复杂数据的效果。



2. 多项式回归

45

当我们要创建适合处理非线性可分数据的模型时，我们需要使用多项式回归。在这种回归技术中，最佳拟合线不是一条直线，而是一条符合数据点的曲线。对于一个多项式回归，一些自变量的指数是大于1的。



多项式回归的几个要点：

- 能够模拟非线性可分的数据;线性回归不能做到这一点。它总体上更灵活，可以模拟一些相当复杂的关系。
- 完全控制要素变量的建模（要设置变量的指数）。
- 需要仔细的设计，需要数据的先验知识才能选择最佳指数。
- 如果指数选择不当，容易过拟合。

2. 多项式回归

46

特征选择

还是以卖房子的事情为例，其实我们前面是说房屋面积和售价之间的关系。但是，实际上我们测量的是房子的长和宽，即房屋前面临街的宽度（frontage）、房屋的纵深（depth）。如下图：

Housing prices prediction

$$h_{\theta}(x) = \theta_0 + \theta_1 \times \text{frontage} + \theta_2 \times \text{depth}$$



如果这样的话，我们死板的套用线性回归方程的话会得到上图左边这样的一个式子。我们让frontage为 x_1 ，让depth为 x_2 。

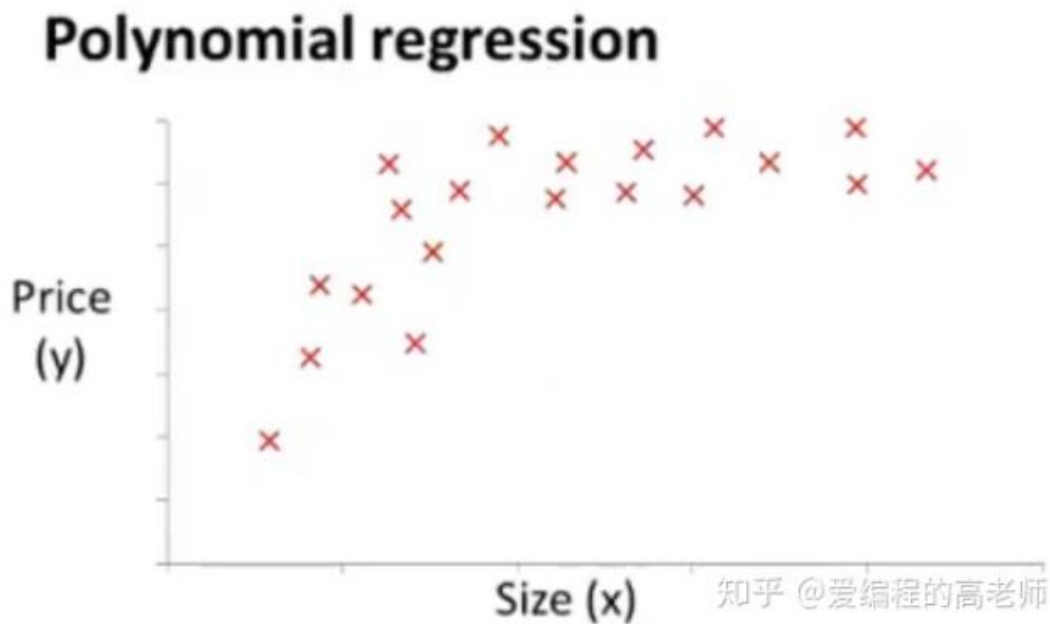
实际上呢，我们是用 $\text{frontage} \times \text{depth} = \text{area}$ 为一个自变量来进行回归分析的，回归方程变为 $h_{\theta}(x) = \theta_0 + \theta_1 x$ 。看上去好很多。

2. 多项式回归

47

多项式回归

我们确定了面积作为一个自变量后，将面积和售价之间的关系绘制一下散点图：



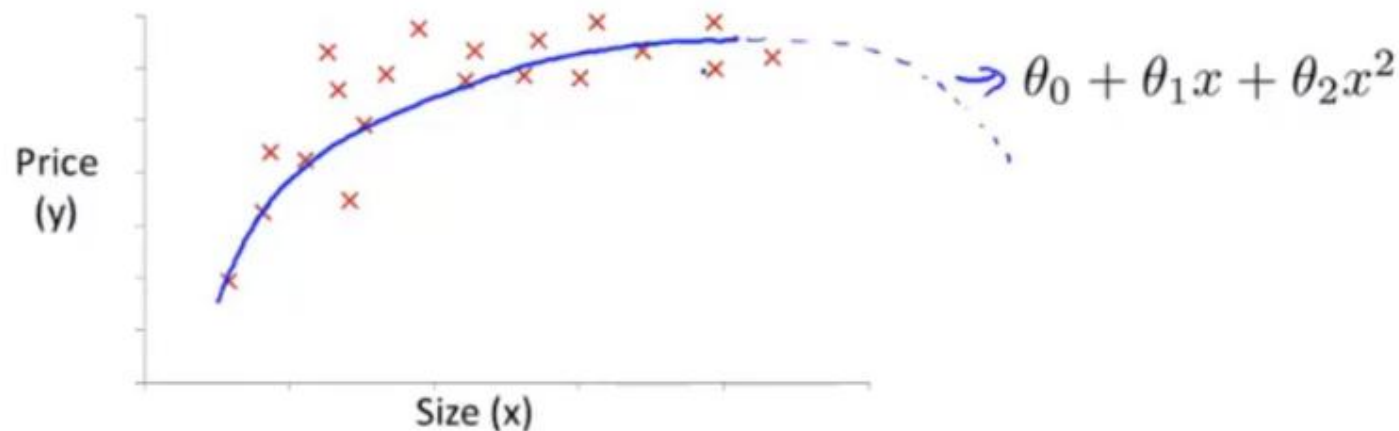
从散点图上一看，这并不是一条直线啊，不能用线性方程来拟合啊，那咋办呢？

这个时候，我们考虑使用二次函数来进行拟合。

2. 多项式回归

48

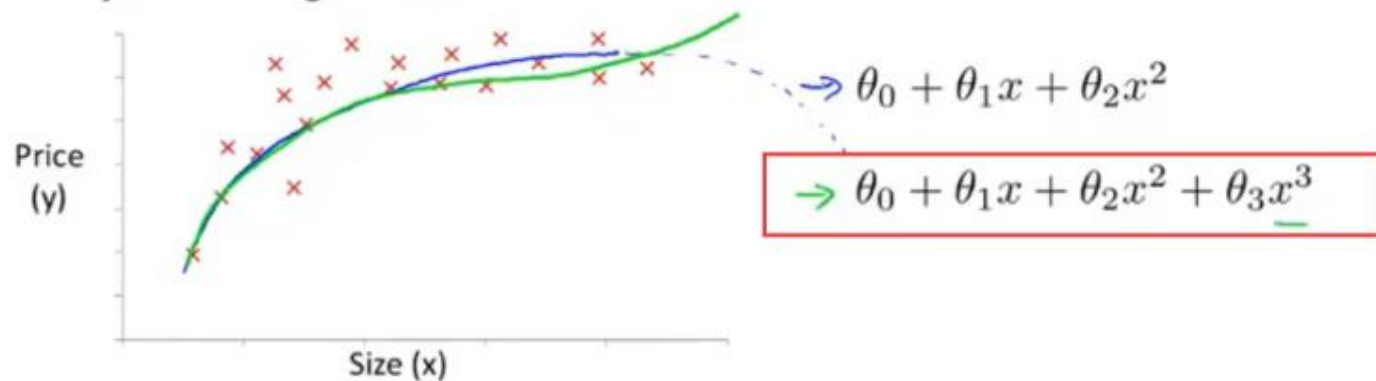
Polynomial regression



这样的二次函数在开头的时候还不错，但是二次函数是一个开口向下的抛物线，它有上升到后面也会有下降，这就不行了，不可能随着房屋面积的增加售价还下降了。看来，二次函数也不行，那还能怎样？

我们考虑使用三次函数，三次函数不会下降，只会上升，如下图：

Polynomial regression



2. 多项式回归

49

如果使用3次方的话，你会发现前面我们讲到过的特征缩放就非常重要，因为：

$$\begin{aligned} h_{\theta}(x) &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 \\ &= \theta_0 + \theta_1(\text{size}) + \theta_2(\text{size})^2 + \theta_3(\text{size})^3 \end{aligned}$$

Size (x)

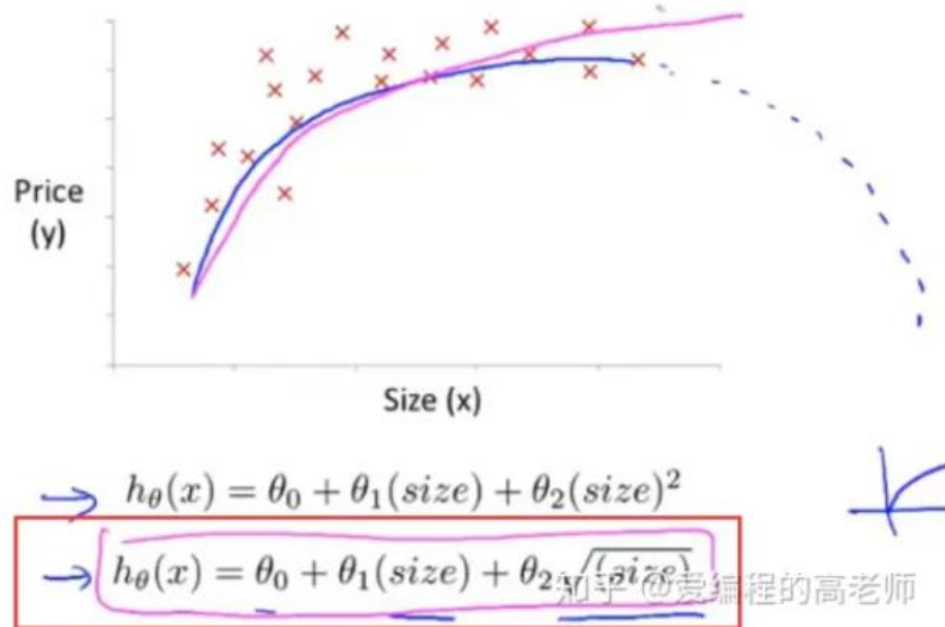
Size: 1-1000
Size²: 1-1,000,000
Size³: 1-1,000,000,000

→ $x_1 = (\text{size})$
→ $x_2 = (\text{size})^2$
→ $x_3 = (\text{size})^3$

如果更细心一点，你会发现其实三次函数也不能很好的拟合最初的那个散点图。因为，随着面积的增加，售价的增长会非常缓慢，和三次函数的形状不一致了。

那这个时候，我们考虑使用下面这样的式子，考虑使用一个根号，如下图。

Choice of features



3 逻辑回归

50

01 分类问题

02 **Sigmoid**函数

03 逻辑回归求解

04 逻辑回归代码实现

1.分类问题

51

01 分类问题

02 Sigmoid函数

03 逻辑回归求解

04 逻辑回归代码实现

监督学习的最主要类型

标签离散

✓ 分类 (Classification)

- ✓ 身高1.85m, 体重100kg的男人穿什么尺码的T恤?
- ✓ 根据肿瘤的体积、患者的年龄来判断良性或恶性?
- ✓ 根据用户的年龄、职业、存款数量来判断信用卡是否会违约?

输入变量可以是离散的, 也可以是连续的。

分类问题

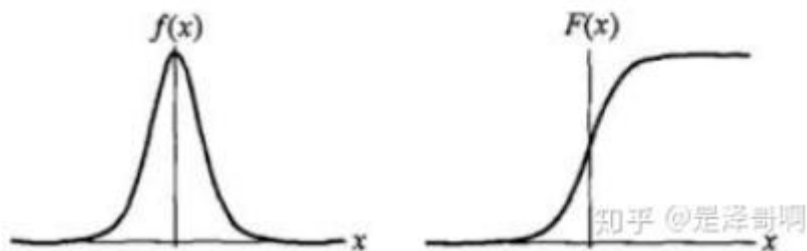
53

- Logistic Regression 虽然被称为回归，但其实际上是分类模型，并常用于二分类。Logistic Regression 因其简单、可并行化、可解释性强深受工业界喜爱。Logistic 回归的本质是：假设数据服从这个分布，然后使用极大似然估计做参数的估计。

Logistic 分布是一种连续型的概率分布，其分布函数和密度函数分别为：

$$F(x) = P(X \leq x) = \frac{1}{1 + e^{-(x-\mu)/\gamma}}$$
$$f(x) = F'(X \leq x) = \frac{e^{-(x-\mu)/\gamma}}{\gamma(1 + e^{-(x-\mu)/\gamma})^2}$$

其中， μ 表示位置参数， $\gamma > 0$ 为形状参数。我们可以看下其图像特征：



Logistic 分布是由其位置和尺度参数定义的连续分布。Logistic 分布的形状与正态分布的形状相似，但是 Logistic 分布的尾部更长，所以我们可以使用 Logistic 分布来建模比正态分布具有更长尾部和更高波峰的数据分布。在深度学习中常用到的 Sigmoid 函数就是 Logistic 的分布函数在 $u = 0, \gamma = 1$ 的特殊形式。

分类问题

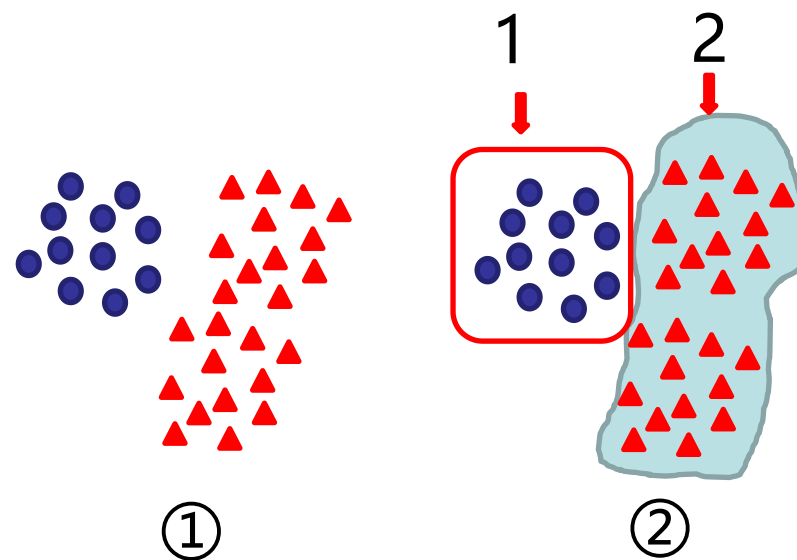
54

二分类

我们先从用蓝色圆形数据定义为类型1，其余数据为类型2；

只需要分类1次

步骤：①->②



二分类

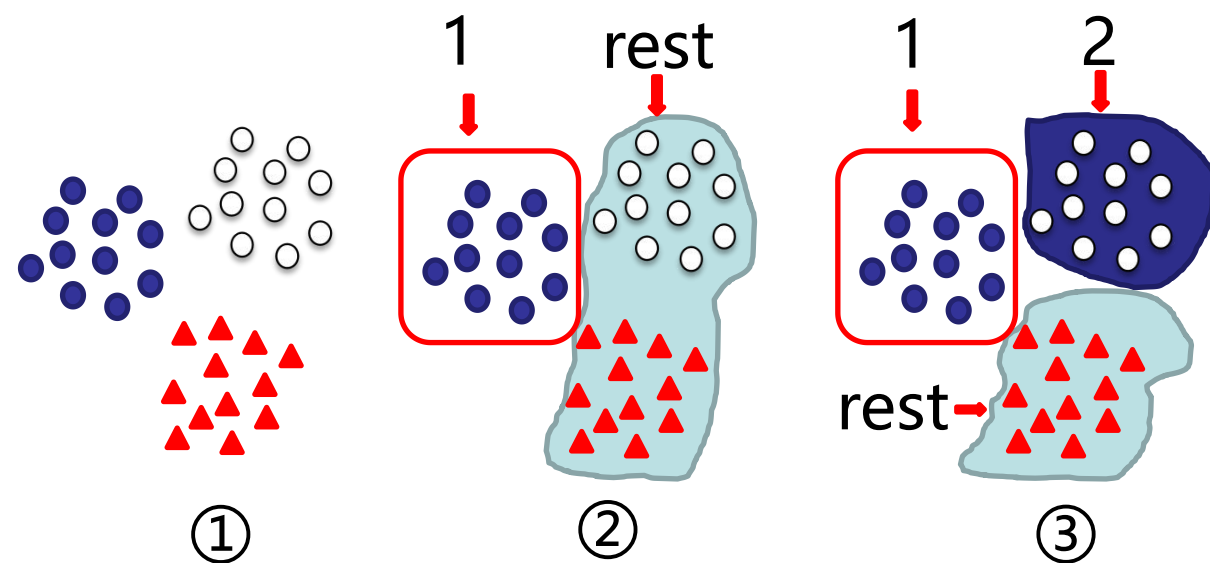
分类问题

55

多分类

我们先定义其中一类为类型1（正类），其余数据为负类（rest）；
接下来去掉类型1数据，剩余部分再次进行二分类，分成类型2和负类；
如果有 n 类，那就需要分类 $n-1$ 次

步骤：①->②->③->.....

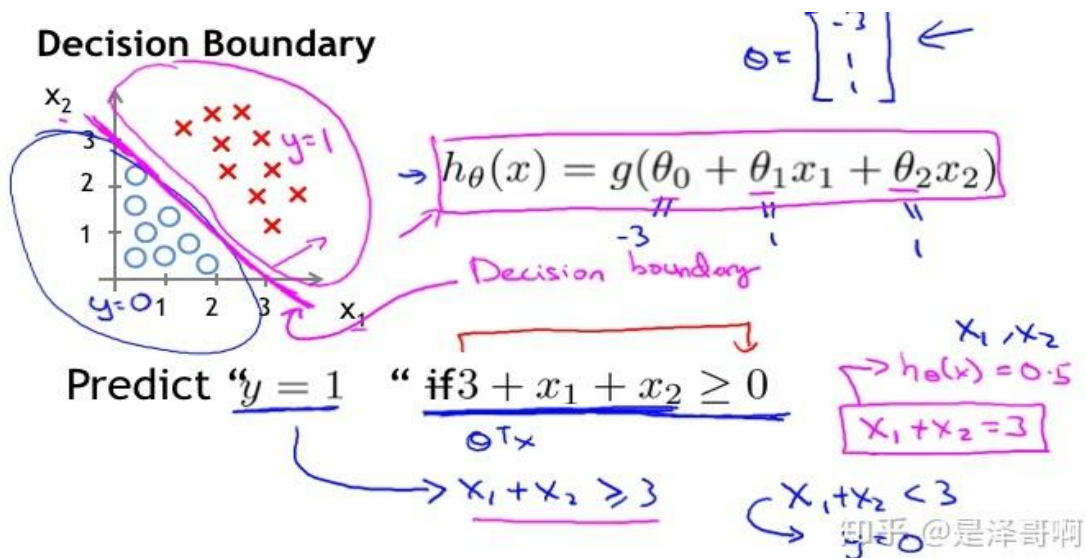


One-vs-All (One-vs-Rest)

一对多 (一对余)

分类问题

56



决策边界可以表示为 $w_1 x_1 + w_2 x_2 + b = 0$ ，假设某个样本点 $h_w(x) = w_1 x_1 + w_2 x_2 + b > 0$ 那么可以判断它的类别为 1，这个过程其实是感知机。

Logistic 回归还需要加一层，它要找到分类概率 $P(Y=1)$ 与输入向量 x 的直接关系，然后通过比较概率值来判断类别。

考虑二分类问题，给定数据集

$$D = (x_1, y_1), (x_2, y_2), \dots, (x_N, y_N), x_i \subseteq R^n, y_i \in 0, 1, i = 1, 2, \dots, N$$

考虑到 $w^T x + b$ 取值是连续的，因此它不能拟合离散变量。可以考虑用它来拟合条件概率 $p(Y = 1|x)$ ，因为概率的取值也是连续的。

但是对于 $w \neq 0$ （若等于零向量则没有什么求解的价值）， $w^T x + b$ 取值为 R ，不符合概率取值为 0 到 1，因此考虑采用广义线性模型。

最理想的是单位阶跃函数：

$$p(y = 1|x) = \begin{cases} 0, & z < 0 \\ 0.5, & z = 0 \\ 1, & z > 0 \end{cases}, \quad z = w^T x + b \rightarrow$$

但是这个阶跃函数不可微，对数几率函数是一个常用的替代函数：

$$y = \frac{1}{1 + e^{-(w^T x + b)}}$$

分类问题

57

于是有：

$$\ln \frac{y}{1-y} = w^T x + b$$

我们将 y 视为 x 为正例的概率，则 $1-y$ 为 x 为其反例的概率。两者的比值称为**几率 (odds)**，指该事件发生与不发生的概率比值，若事件发生的**概率**为 p 。则对数几率：

$$\ln(odds) = \ln \frac{y}{1-y}$$

将 y 视为类后验概率估计，重写公式有：

$$w^T x + b = \ln \frac{P(Y = 1|x)}{1 - P(Y = 1|x)}$$
$$P(Y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

也就是说，输出 $Y=1$ 的对数几率是由输入 x 的**线性函数**表示的模型，这就是**逻辑回归模型**。当 $w^T x + b$ 的值越接近正无穷， $P(Y = 1|x)$ 概率值也就越接近 1。因此**逻辑回归的思路**是，先拟合决策边界(不局限于线性，还可以是多项式)，再建立这个边界与分类的概率联系，从而得到了二分类情况下的概率。

2.Sigmoid函数

58

01 分类问题

02 Sigmoid函数

03 逻辑回归求解

04 逻辑回归代码实现

2.Sigmoid函数

59

线性回归的函数 $h(x) = z = w^T x$, 范围是 $(-\infty, +\infty)$ 。

而分类预测结果需要得到 $[0,1]$ 的概率值。

在二分类模型中, 事件的几率odds: 事件发生与事件不发生的概率之比为 $\frac{p}{1-p}$,

称为事件的发生比 (the odds of experiencing an event)

其中 p 为随机事件发生的概率, p 的范围为 $[0,1]$ 。

取对数得到: $\log \frac{p}{1-p}$, 而 $\log \frac{p}{1-p} = w^T x = z$

求解得到: $p = \frac{1}{1+e^{-w^T x}} = \frac{1}{1+e^{-z}}$

2.Sigmoid函数

60

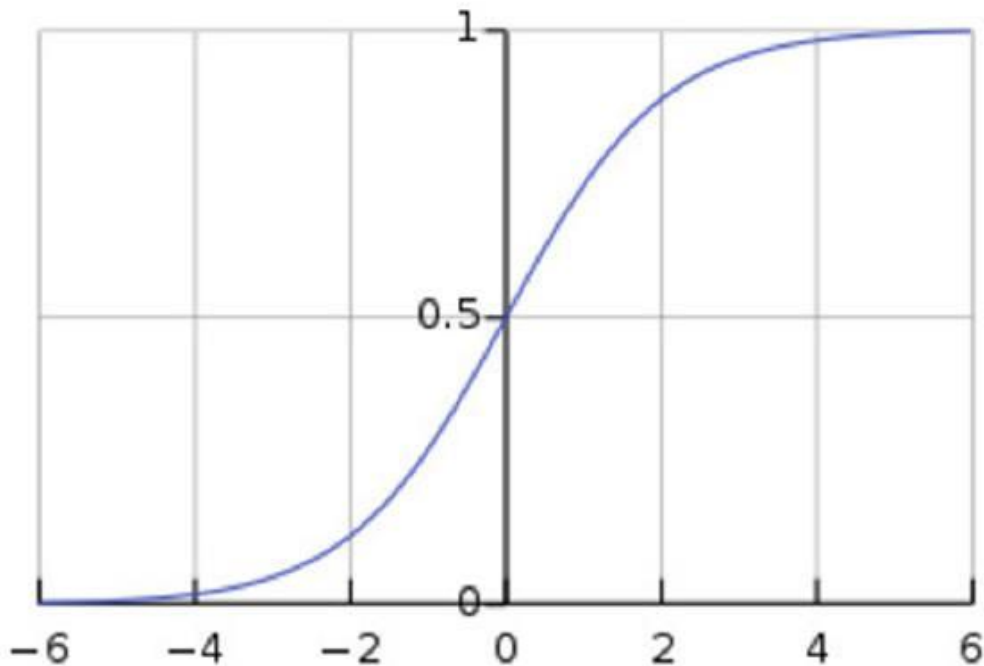
Sigmoid 函数

$\sigma(z)$ 代表一个常用的逻辑函数 (logistic function) 为S形函数 (Sigmoid function)

$$\text{则: } \sigma(z) = g(z) = \frac{1}{1+e^{-z}} \quad z=w^T x + b$$

合起来, 我们得到逻辑回归模型的损失函数:

$$L(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$



当 $\sigma(z)$ 大于等于0.5时, 预测 $y=1$

当 $\sigma(z)$ 小于0.5时, 预测 $y=0$

注意: 若表达式 $h(x) = z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w^T x + b$, 则 b 可以融入到 w_0 , 即: $z=w^T x$

2.Sigmoid函数

61

$$\hat{y} = \begin{cases} 1, & \hat{p} \geq 0.5 \\ 0, & \hat{p} \leq 0.5 \end{cases} \quad cost \begin{cases} \text{如果 } y = 1, p \text{ 越小, } cost \text{ 越大} \\ \text{如果 } y = 0, p \text{ 越大, } cost \text{ 越大} \end{cases}$$

接下来就是数学最美的地方了，把上面的描述转换为数学公式：

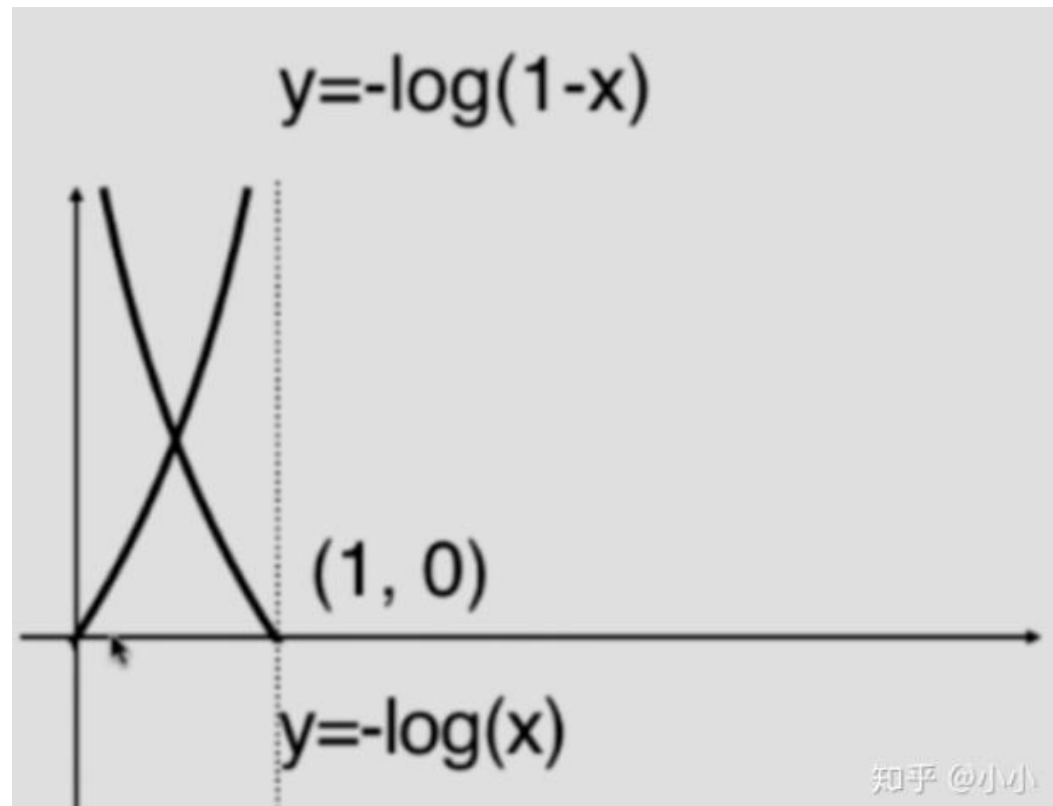
$$cost \begin{cases} -\log(\hat{y}) & \text{if } y = 1 \\ -\log(1 - \hat{y}) & \text{if } y = 0 \end{cases}$$

那有了这么一个函数，但如何把他们整合到一起呢？

$$cost = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

到此，就可以总结出Logistic Regression的损失函数：

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(\hat{y}) + (1 - y^{(i)}) \log(1 - \hat{y}))$$



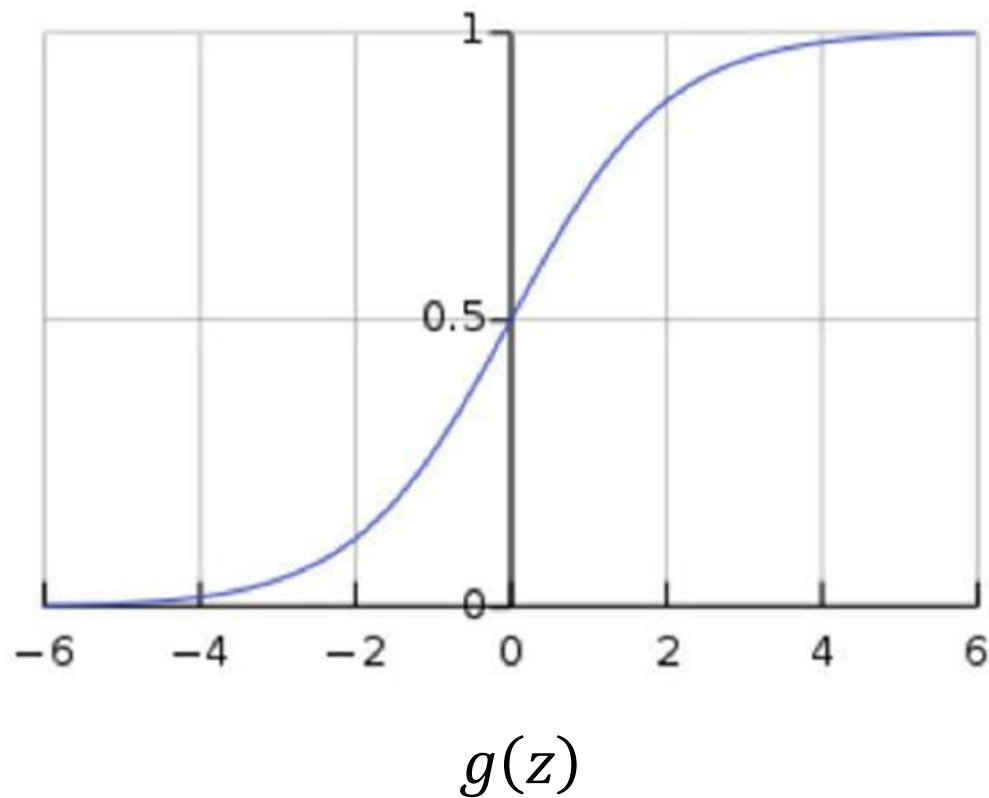
知乎 @小小

2.Sigmoid函数

62

将 z 进行逻辑变换: $g(z) = \frac{1}{1+e^{-z}}$

$$\begin{aligned} g'(z) &= \left(\frac{1}{1+e^{-z}} \right)' \\ &= \frac{e^{-z}}{(1+e^{-z})^2} \\ &= \frac{1+e^{-z}-1}{(1+e^{-z})^2} \\ &= \frac{1}{(1+e^{-z})} \left(1 - \frac{1}{(1+e^{-z})} \right) \\ &= g(z)(1-g(z)) \end{aligned}$$



3.逻辑回归求解

63

01 分类问题

02 Sigmoid函数

03 逻辑回归求解

04 逻辑回归代码实现

3.逻辑回归求解

64

假设一个二分类模型：

$$p(y = 1|x; w) = h(x)$$

$$p(y = 0|x; w) = 1 - h(x)$$

则：

$$p(y|x; w) = (h(x))^y (1 - h(x))^{1-y}$$

逻辑回归模型的假设是： $h(x) = g(w^T x) = g(z)$

其中 $z = w^T x$, 逻辑函数 (**logistic function**)公式为：

$$g(z) = \frac{1}{1+e^{-z}}, \quad g'(z) = g(z)(1 - g(z))$$

3.逻辑回归求解

65

损失函数

$$L(\hat{y}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

$\hat{y}$ 表示预测值 $h(x)$

y 表示真实值

为了衡量算法在全部训练样本上的表现如何，我们需要定义一个算法的代价函数，算法的代价函数是对 m 个样本的损失函数求和然后除以 m ：

代价函数

$$J(w) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{m} \sum_{i=1}^m \left(-y^{(i)} \log \hat{y}^{(i)} - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right)$$

3.逻辑回归求解

66

求解过程：

似然函数为： $L(w) = \prod_{i=1}^m P(y^{(i)} | x^{(i)}; w) = \prod_{i=1}^m (h(x^{(i)}))^{y^{(i)}} (1 - h(x^{(i)}))^{1-y^{(i)}}$

似然函数两边取对数，则连乘号变成了连加号：

$$l(w) = \log L(w) = \sum_{i=1}^m (y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))$$

代价函数为：

$$J(w) = -\frac{1}{m} l(w) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))$$

3.逻辑回归求解

67

梯度下降求解过程：

$$w_j := w_j - \alpha \frac{\partial J(w)}{\partial w}$$

$$J(w) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))$$

$$\frac{\partial}{\partial w_j} J(w) = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\text{则： } w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

3.逻辑回归求解

68

求解过程： $\frac{\partial}{\partial w_j} J(w) = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$ 的推导过程：

$$J(w) = -\frac{1}{m} \sum_{i=1}^m \underline{(y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))}$$



$$\begin{aligned} & y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})) \\ &= y^{(i)} \log\left(\frac{1}{1 + e^{-w^T x^{(i)}}}\right) + (1 - y^{(i)}) \log\left(1 - \frac{1}{1 + e^{-w^T x^{(i)}}}\right) \\ &= -y^{(i)} \log(1 + e^{-w^T x^{(i)}}) - (1 - y^{(i)}) \log(1 + e^{w^T x^{(i)}}) \end{aligned}$$

3.逻辑回归求解

69

求解过程： $\frac{\partial}{\partial w_j} J(w) = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$ 的推导过程：

$$\begin{aligned} \frac{\partial}{\partial w_j} J(w) &= \frac{\partial}{\partial w_j} \left(-\frac{1}{m} \sum_{i=1}^m \left(-y^{(i)} \log(1 + e^{-w^T x^{(i)}}) - (1 - y^{(i)}) \log(1 + e^{w^T x^{(i)}}) \right) \right) \\ &= -\frac{1}{m} \sum_{i=1}^m \left(-y^{(i)} \frac{-x_j^{(i)} e^{-w^T x^{(i)}}}{1 + e^{-w^T x^{(i)}}} - (1 - y^{(i)}) \frac{x_j^{(i)} e^{w^T x^{(i)}}}{1 + e^{w^T x^{(i)}}} \right) \\ &= -\frac{1}{m} \sum_{i=1}^m (y^{(i)} - h(x^{(i)})) x_j^{(i)} \\ &= \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)} \end{aligned}$$

3.逻辑回归求解

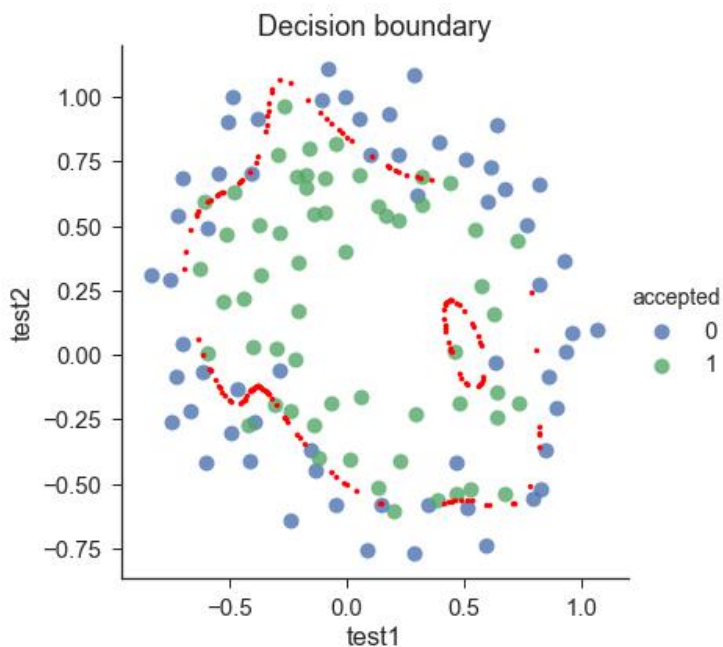
70

正则化：目的是为了**防止过拟合**

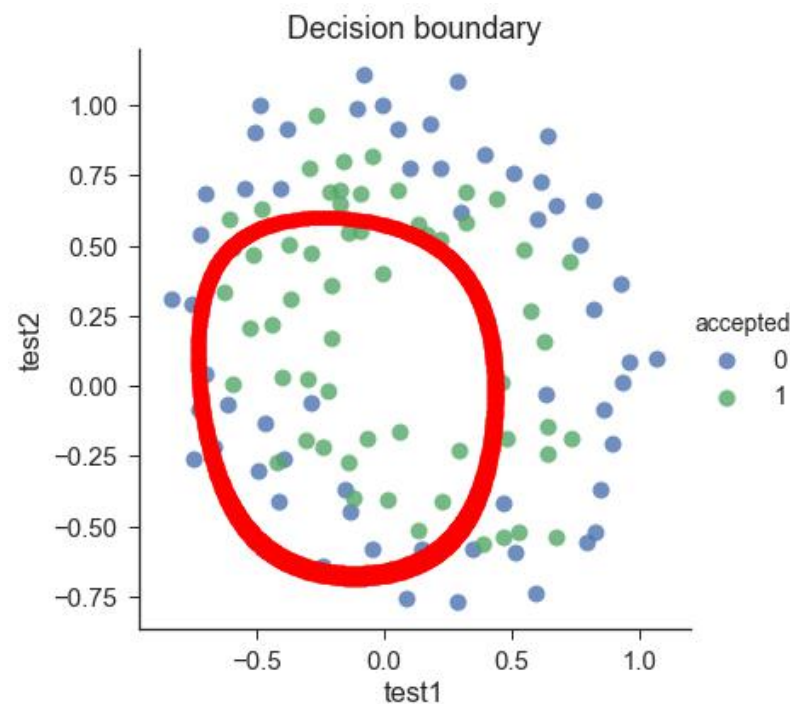
正则化项

$$J(w) = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h(x^{(i)})) - (1 - y^{(i)}) \log(1 - h(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

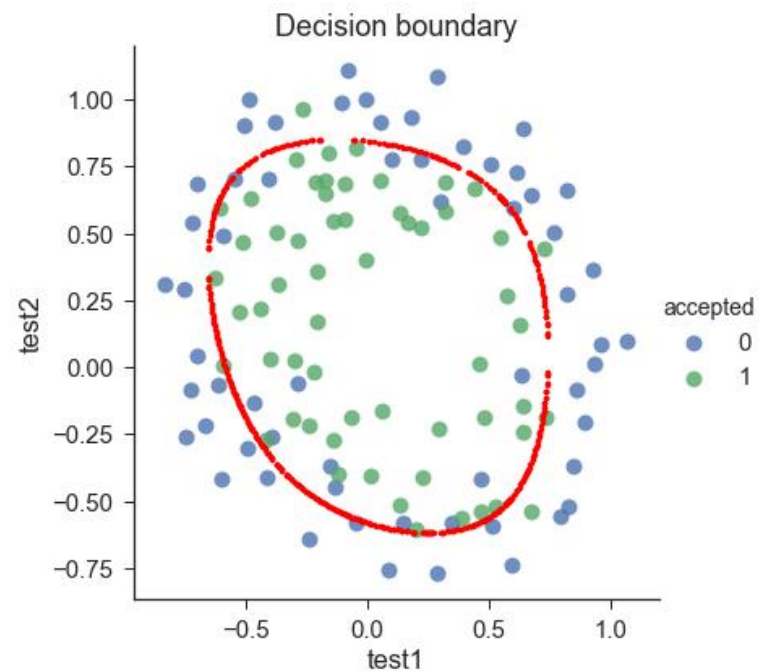
当 λ 的值开始上升时，降低了方差。



没有正则化，过拟合



正则化过度，欠拟合



适当的正则化

4.逻辑回归代码实现

71

01 分类问题

02 Sigmoid函数

03 逻辑回归求解

04 逻辑回归代码实现

4.逻辑回归代码实现

72

Sigmoid 函数

$$\sigma(z) = g(z) = \frac{1}{1+e^{-z}}$$

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))
```

代价函数

$$J(w) = -\frac{1}{m} \sum_{i=1}^m (y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)})))$$

```
def cost(w, X, y):  
    w = np.matrix(w)  
    X = np.matrix(X)  
    y = np.matrix(y)  
    first = np.multiply(-y, np.log(sigmoid(X * w.T)))  
    second = np.multiply((1 - y), np.log(1 - sigmoid(X * w.T)))  
    return np.sum(first - second) / (len(X))
```

4.逻辑回归代码实现

73

正则化

$$J(w) = \frac{1}{m} \sum_{i=1}^m \left[-y^{(i)} \log(h(x^{(i)})) - (1 - y^{(i)}) \log(1 - h(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

```
def costReg(w, X, y, LearningRate):  
    w = np.matrix(w)  
    X = np.matrix(X)  
    y = np.matrix(y)  
    first = np.multiply(-y, np.log(sigmoid(X * w.T)))  
    second = np.multiply((1 - y), np.log(1 - sigmoid(X * w.T)))  
    reg = (learningRate /  
           (2 * len(X))) * np.sum(np.power(w[:, 1:w.shape[1]], 2))  
    return np.sum(first - second) / len(X) + reg
```

- [1] HOSMER D W, LEMESHOW S, STURDIVANT R X. Applied logistic regression[M]. Wiley New York.2000.
- [2] Andrew Ng. Machine Learning[EB/OL]. Stanford University,2014.
<https://www.coursera.org/course/ml>
- [3] 李航. 统计学习方法[M]. 清华大学出版社,2019.
- [4] Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning[M]. Springer, New York, NY, 2001.
- [5] CHRISTOPHER M. BISHOP. Pattern Recognition and Machine Learning[M]. Springer,2006.
- [6] Stephen Boyd, Lieven Vandenberghe, Convex Optimization[M]. Cambridge University Press, 2004.
- [7] TIBSHIRANI R. Regression selection and shrinkage via the lasso[J]. Journal of the Royal Statistical Society Series B, 1996, 58(1): 267–288.

A photograph of a modern, multi-story building at night. The building's facade is illuminated with horizontal bands of warm yellow light. The foreground is a dark, blue-tinted area with some architectural elements and a few parked cars. The overall atmosphere is serene and modern.

谢谢!